

**AUTOSCORE-KABADDI: AUTOMATED KABADDI
SCORING USING MULTI-CAMERA PLAYER
TRACKING AND INTERACTION REASONING**

A PROJECT REPORT

Submitted by

SRILEKHA RAMKUMAR	2022115076
MYTREYAN JP	2022115102
THIYANESHWAR T	2022115136
SUGANTH S A	2022115308

submitted to the faculty of

INFORMATION AND COMMUNICATION ENGINEERING

in partial fulfillment

for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY

CHENNAI 600 025

APRIL 2026

ANNA UNIVERSITY
CHENNAI - 600 025
BONAFIDE CERTIFICATE

Certified that this project report titled **AUTOSCORE-KABADDI: AUTOMATED KABADDI SCORING USING MULTI-CAMERA PLAYER TRACKING AND INTERACTION REASONING** is the bonafide work of **SRILEKHA RAMKUMAR (2022115076)**, **MYTREYAN JP (2022115102)**, **THIYANESHWAR T (2022115136)** and **SUGANTH S A (2022115308)** who carried out project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE : CHENNAI

DATE :

Dr. D. NARASHIMAN
TEACHING FELLOW
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI - 600025

COUNTERSIGNED

Dr. P. YOGESH

HEAD OF THE DEPARTMENT

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY

CHENNAI 600025

ABSTRACT

Kabaddi is a fast-paced contact sport where scoring decisions depend on accurately interpreting complex player interactions in real time. Manual officiating faces challenges from severe occlusions during tackles, rapid movements, and limited viewing angles, leading to inconsistencies and delayed decisions. Unlike football and cricket, which benefit from technologies like VAR and Hawk-Eye, Kabaddi lacks automated vision-based scoring infrastructure. This project proposes **AUTOSCORE-KABADDI**, an intelligent multi-camera vision system that analyzes synchronized match footage to detect, track, and interpret player actions according to official Kabaddi rules.

The system employs occlusion-robust tracking to maintain player identities during dense interactions, transforms coordinates into structured raid intelligence through bird's-eye projections and interaction graphs, and validates scoring events including touches, tackles, bonus points, and boundary violations using rule-aware logic. It provides immersive visualization with 3D reprojections, skeletal animations, decision dashboards, and synchronized scoreboard overlays.

By integrating computer vision, graph-based modeling, and probabilistic reasoning, AUTOSCORE-KABADDI delivers a transparent, real-time decision-support framework for professional officiating, performance analytics, and tactical evaluation, enhancing accuracy, fairness, and transparency while providing structured metadata for coaching insights.

ACKNOWLEDGEMENT

It is our privilege to express our deepest sense of gratitude and sincere thanks to **Dr. D. NARASHIMAN**, Teaching fellow, Project Guide, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, for his constant supervision, encouragement, and support in our project work. We greatly appreciate the constructive advice and motivation that was given to help us advance our project in the right direction.

We are grateful to **Dr. P. YOGESH**, Professor and Head of the Department, Information Science and Technology, College of Engineering, Guindy, Anna University for providing us with the opportunity and necessary resources to do this project.

We extend our heartfelt thanks to the project committee members, **Dr. M. VIJAYALAKSHMI**, Professor, **Dr. ABIRAMI MURUGAPPAN**, Professor, and **Dr. N. THANGARAJ**, Associate Professor and for their constructive feedback and valuable suggestions that enriched our work. We also thank the faculty member and non teaching staff members of the Department of Information Science and Technology, Anna University, Chennai for their valuable support throughout the course of our project work.

SRILEKHA RAMKUMAR
MYTREYAN JP
THIYANESHWAR T
SUGANTH S A

TABLE OF CONTENTS

ABSTRACT	iii
ACKNOWLEDGEMENT	iv
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
1 INTRODUCTION	1
1.1 PROJECT BACKGROUND	1
1.1.1 Complexity of Kabaddi scoring and game dynamics	2
1.1.2 Advancements in Computer Vision for Sports Analytics	3
1.2 PROBLEM STATEMENT	4
1.3 PROPOSED SYSTEM	4
1.4 OBJECTIVES OF THE PROPOSED SYSTEM	6
1.5 ORGANIZATION OF CHAPTERS	7
2 LITERATURE SURVEY	9
2.1 OCCLUSION-ROBUST MULTI-OBJECT TRACKING	9
2.2 GROUP DETECTION AND GRAPH-BASED SPATIAL MODELING	10
2.3 HUMAN-HUMAN INTERACTION DETECTION IN SPORTS	11
2.4 FACTOR GRAPH-BASED GROUP BY ACTIVITY	11
2.5 SUMMARY AND RESEARCH GAP	13
2.6 CHALLENGES	14
3 SYSTEM ARCHITECTURE	15
3.1 MULTI-OBJECT TRACKING IN OCCLUSION	15
3.1.1 Multi-Camera Synchronization And Acquisition	15
3.1.2 Geometric Calibration And Court Normalization	15
3.1.3 Player Detection And Tracking Initialization	17
3.1.4 Identity Retention Through Spatial Group Modeling	17
3.1.5 Occlusion Handling And PRedictive Motion Estimation	18
3.1.6 Non-Rigid Motion Refinement	18
3.2 INTERACTION ANALYSIS AND SCORING LOGIC	18

3.2.1	Bird's-Eye Coordinate Transformation	20
3.2.2	Pairwise Interaction Proposal	20
3.2.3	Active Factor Graph Network Construction	20
3.2.4	Multi-Level Factor Node Construction	21
3.2.5	Graph-Based Pressure Scoring And Inference	21
3.2.6	Temporal Interaction Validation And Confidence Fusion	22
3.2.7	Kabaddi-Specific Reasoning Engine	23
3.2.8	Visual Confirmation Through Touch Classification	24
3.2.9	Comprehensive Raid Metadata and Event Logging	24
3.3	VISUALIZATION AND REPLAY RECONSTRUCTION	25
3.3.1	3D Coordinate Reprojection	26
3.3.2	Interactive Graph Visualization and Decision Dashboard	26
3.3.3	Dynamic Scoreboard Overlay Generation	27
3.3.4	Immersive 3D Replay Rendering	28
4	IMPLEMENTATION	29
4.1	MULTI-OBJECT TRACKING UNDER OCCLUSION	29
4.1.1	Yolov8-Based Player Detection	29
4.1.2	Kalman Filter Motion Prediction With Optical Flow Compensation	30
4.1.3	Hungarian Algorithm For Tracking-Detection Association	31
4.1.4	Score-Based Gallery Management	33
4.1.5	Homography-Based Court Normalization	34
4.2	ACTIVE FACTOR GRAPH NETWORK AND INTERACTION REASONING	36
4.2.1	Interaction Proposal Generation	36
4.2.2	Automatic Raider Identification	37
4.2.3	Temporal Interaction Validation	39
4.2.4	Active Factor Graph Network Construction	39
4.2.5	Third-Order Factor Node Construction	41
4.2.6	Kabaddi AFGN Reasoning and Scoring	42
4.2.7	Visual Confirmation Through Touch Classification	44
4.3	CONFIRMED EVENT ARCHIVAL AND DATASET GENERATION	45
4.3.1	Event Window Extraction And Buffering	45

4.3.2	Dataset Export With Pose Annotation	46
4.4	LIVE VISUALIZATION AND API SERVER	47
4.4.1	Fast-API Backend Architecture	47
4.4.2	State Synchronization And Event Access	48
4.4.3	Visualization Rendering	49
5	RESULTS AND DISCUSSIONS	50
5.1	TRACKING RESULTS	50
5.1.1	Tracking Accuracy	50
5.1.2	Occlusion Handling	51
5.1.3	Court Mapping Results	51
5.2	INTERACTION REASONING RESULTS	52
5.2.1	Interaction Detection	52
5.2.2	Factor Graph Construction	54
5.2.3	Event Detection Results	54
5.3	TOUCH CLASSIFICATION RESULTS	55
5.3.1	Classifier Performance	55
5.3.2	Hybrid Validation	55
5.4	DASHBOARD VISUALIZATION	56
5.4.1	Interaction Graph	56
5.4.2	Event Timeline	57
5.4.3	Team Analytics	58
5.4.4	Live Scoreboard	59
6	CONCLUSION	60
	REFERENCE	62

LIST OF FIGURES

3.1	Overall System Architecture: Three-Module Pipeline from Multi-Camera Input to Visualization Output	16
3.2	Interaction Logic Engine with Graph-Based Reasoning and Rule Validation	19
5.1	Score-based gallery management	51
5.2	Live tracking visualization	52
5.3	Human-Human Interaction (HHI) event detection	53
5.4	Human-Line Interaction (HLI) event detection	53
5.5	Touch classification system in action	56
5.6	2D interaction graph and AFGN structure	57
5.7	Confirmed events dashboard	58
5.8	Team-based action tracking interface	59
5.9	Live scoreboard display showing real-time scores	59

LIST OF ABBREVIATIONS

AFGN	Active Factor Graph Network
DeepSORT	Deep Simple Online and Realtime Tracking
EPnP	Efficient Perspective-n-Point
HHI	Human Human Interaction
OMC	Optical Flow-based Motion Compensation
Re-ID	Re-Identification
SGM	Score-based Gallery Management
UI	User Interface
VAR	Video Assistant Referee
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

This chapter introduces the AUTOSCORE-KABADDI system by outlining the background, challenges, and motivation behind automated Kabaddi scoring. It highlights the limitations of manual officiating and the lack of technological support in the sport. The chapter presents the problem statement and proposes a vision-based solution leveraging computer vision and graph-based reasoning. It also defines the key objectives of the system and provides an overview of the overall architecture and workflow. Finally, it outlines the organization of the report to guide the reader through subsequent chapters.

1.1 PROJECT BACKGROUND

Kabaddi is a fast-paced, high-contact sport that demands precise coordination between players, referees, and officials. Despite its growing international recognition and professional league structures, the scoring and officiating process in Kabaddi continues to rely heavily on manual human judgment. Referees must simultaneously monitor multiple players, track complex interactions, validate rule compliance, and make real-time decisions under intense physical and temporal pressure. This often results in inconsistencies, delayed decisions, and occasional disputes.

Unlike sports such as football, basketball, or cricket, where technologies like Hawk-Eye, VAR systems, player tracking sensors, and automated event detection assist officials, Kabaddi lacks a widely adopted automated vision-based scoring infrastructure. Kabaddi scoring involves

interpreting touches, tackles, line crossings, bonus points, super tackles, and all-outs directly from dynamic player interactions in real time. The complexity of these events makes purely manual evaluation prone to human error.

With advancements in computer vision, deep learning, multi-object tracking, and interaction reasoning, it is now feasible to design a system capable of automatically interpreting Kabaddi rules directly from match video. The proposed system, **AUTOSCORE-KABADDI**, introduces an intelligent, multi-camera, vision-based scoring framework that aims to improve accuracy, transparency, and scalability in Kabaddi officiating and analytics.

1.1.1 Complexity of Kabaddi scoring and game dynamics

Kabaddi scoring is inherently interaction-driven, as a single raid may involve multiple defenders, rapid direction changes, partial body contacts, and boundary decisions occurring within seconds. Scoring is achieved through a mix of offensive agility and defensive coordination: a raider earns points by touching one or more defenders and returning safely to their half, while they can also secure a bonus point by crossing the bonus line when at least six defenders are present on the mat.

Conversely, the defending team scores by executing a successful tackle that prevents the raider from reaching the midline. Special circumstances like a *Super Tackle*, which occurs when three or fewer defenders successfully stop a raider, offer additional scoring opportunities. Finally, the game accounts for *All-Out* conditions where an entire team is eliminated, alongside technical faults that may grant points to the opposition for rule violations.

Each of these scoring situations requires accurate interpretation of spatial positioning, player identity, temporal continuity, and rule validation. Dense tackles often create extreme occlusion, where multiple players overlap visually, making it difficult to maintain identity tracking. Additionally, sudden non-rigid movements such as dives, lunges, and rapid directional shifts challenge traditional motion models.

Therefore, an automated scoring system must go beyond simple object detection. It must integrate robust tracking, interaction modeling, contextual reasoning, and rule validation mechanisms capable of understanding group dynamics in real time.

1.1.2 Advancements in Computer Vision for Sports Analytics

Recent developments in computer vision and deep learning have significantly enhanced multi-object tracking and group interaction recognition. Modern object detection models such as YOLO-based architectures allow accurate player detection in real time. Tracking algorithms such as DeepSORT and Kalman filtering enable identity persistence across frames. Furthermore, advanced reasoning models such as graph-based interaction networks and factor graph reasoning engines provide the ability to interpret higher-order interactions between multiple individuals.

Multi-camera geometry techniques, including homography estimation and epipolar geometry, allow transformation of distorted broadcast views into standardized court-aligned projections. Spatio-temporal reasoning frameworks can analyze motion sequences and validate actions against contextual rules.

These advancements collectively create the technological foundation necessary to design an automated Kabaddi scoring system capable of interpreting complex match events directly from video data.

1.2 PROBLEM STATEMENT

Current Kabaddi officiating relies entirely on manual human judgment to interpret scoring events in real time, leading to inconsistent decisions, subjective interpretations, and limited transparency during critical match situations. There is no automated system capable of analyzing multi-camera match footage to detect, track, and validate scoring actions such as touches, tackles, bonus points, super tackles, and boundary violations according to official rules. The absence of such technology restricts objective decision verification, limits post-match analysis, and hinders the development of data-driven performance insights.

An end-to-end vision-based system is required to automatically interpret Kabaddi matches from synchronized video feeds by maintaining player identities during dense interactions, reasoning about multi-player spatial configurations and contact events, validating scoring decisions based on rule constraints and game state, and generating transparent real-time outputs for officiating support and tactical analysis.

1.3 PROPOSED SYSTEM

The proposed system, **AUTOSCORE-KABADDI: Automated Kabaddi Scoring via Multi-Camera Player Tracking and Interaction Reasoning**, introduces a comprehensive three-module framework designed to interpret Kabaddi matches directly from synchronized multi-camera video streams.

MULTI-OBJECT TRACKING IN OCCLUSION

The first module focuses on robust player detection and identity preservation under extreme occlusion. Raw multi-camera feeds undergo geometric preprocessing to correct distortions and align views into an observable court perspective.

A Score-based Gallery Management (SGM) mechanism maintains reliable player embeddings to prevent identity switches. Occlusion awareness is enhanced using group-context reasoning combined with Kalman filtering to reduce inconsistencies during dense tackles. Optical Flow Motion Compensation (OMC) further refines tracking during rapid non-rigid movements such as dives and lunges. This module outputs temporally consistent player positions and identities, even during high-density physical interactions.

INTERACTION ANALYSIS AND SCORING LOGIC

The second module transforms tracked player coordinates into a structured reasoning pipeline. Using Bird's-Eye projection through homography estimation, player positions are mapped onto a standardized court model.

The Interaction Logic Engine analyzes potential human-human interactions through pairwise interaction proposal generation, the construction of dynamic interaction graphs, and factor graph reasoning for higher-order consistency, all while maintaining spatio-temporal context validation. This module determines the type of interaction (touch, tackle, block, bonus attempt, etc.) and validates it against Kabaddi scoring rules using a logic validation engine. The output includes structured raid analysis, event classification, and score metadata generation.

IMMERSIVE TACTICAL VISUALIZATION

The third module converts analytical outputs into immersive visual representations. Using 3D coordinate reprojection techniques such as EPnP and keypoint-driven animation via inverse kinematics (FABRIK), player movements are reconstructed in a 3D tactical environment.

This module enables reconstructed 3D replays, interactive decision dashboards, transparent score overlays, and tactical visualization for coaching and analysis. Together, these modules form an end-to-end intelligent officiating and analytics system that bridges raw video input and structured scoring decisions.

1.4 OBJECTIVES OF THE PROPOSED SYSTEM

The overall aim of this project is to design and develop a fully automated, **Vision-based Scoring System** capable of interpreting Kabaddi rules directly from multi-camera match footage using computer vision and graph-based reasoning.

The specific objectives are as follows:

1. Develop a YOLOv8-based tracking pipeline with Score-based Gallery Management (SGM) and Optical Flow Motion Compensation (OMC) to achieve 85% identity consistency during occlusions using dual-anchor appearance modeling and adaptive Kalman filtering.
2. Design an Active Factor Graph Network (AFGN) with first-order pair factors, second-order line factors, and third-order containment

factors, using message-passing inference for spatial-temporal validation and achieving 60–70% interaction classification accuracy.

3. Implement a Kabaddi-specific reasoning engine to interpret raid progression, contact events, defender actions, and boundary violations using confidence-weighted inference and rule-based constraints.
4. Establish a homography-based transformation pipeline to map broadcast views to a standardized $10\text{m} \times 6.5\text{m}$ court with real-time foot position projection for accurate spatial reasoning.
5. Develop a multi-panel visualization framework with broadcast overlays, bird's-eye projection and real-time event logs .

1.5 ORGANIZATION OF CHAPTERS

The structure of this report is organized in a systematic and coherent manner to effectively present the proposed work.

Chapter 1 introduces the project by presenting the background of kabaddi scoring complexity, the motivation for automated vision-based analysis, the problem statement, objectives, solution overview, and the overall system architecture.

Chapter 2 provides a review of existing literature related to multi-object tracking under occlusion, graph-based interaction recognition, Active Factor Graph Networks, sports video analytics, deep learning-based action recognition, and rule-based reasoning systems.

Chapter 3 explains the system architecture and design of the proposed framework, including the Multi-Object Tracking module,

Homography-based Court Mapping, Interaction Proposal Engine, Active Factor Graph Network, Temporal Validation Manager, Kabaddi AFGN Reasoning Engine, and Visual Confirmation Bridge.

Chapter 4 describes the implementation of the system, covering YOLOv8 detection integration, score-based gallery management, optical flow compensation, factor graph construction, temporal candidate management, rule validation logic, ResNet18 touch classifier, and dataset export pipeline.

Chapter 5 presents the results and discussion, including system evaluation, tracking consistency metrics, interaction classification accuracy, scoring validation performance, classifier effectiveness, real-time processing benchmarks, and outputs from different stages of the analysis pipeline.

References lists all the sources and materials referred to in this project.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews recent advancements in occlusion-robust tracking, group detection in crowded environments, human-human interaction modeling, and factor graph-based reasoning frameworks that directly influence the design of the proposed AUTOSCORE-KABADDI system.

2.1 OCCLUSION-ROBUST MULTI-OBJECT TRACKING

Reliable player tracking is the foundation of any automated sports analysis system. In Kabaddi, extreme physical occlusion occurs during tackles, chain holds, and defensive formations, making identity preservation particularly challenging.

Hong et al.[1] proposed an occlusion-robust multi-object tracking framework that integrates Score-based Gallery Management (SGM) and Optical Flow-based Motion Compensation (OMC). Traditional Re-ID-based trackers update appearance embeddings frame-by-frame, often incorporating noisy embeddings during occlusion, which leads to identity switches. SGM addresses this issue by maintaining a curated embedding gallery, updating embeddings only when similarity thresholds are satisfied. This selective embedding strategy significantly reduces identity drift.

In addition, the authors integrate Optical Flow-based Motion Compensation (OMC) to enhance Kalman Filter predictions in non-rigid motion scenarios. Since constant velocity assumptions often fail in sports environments,

optical flow cues refine displacement estimation at the pixel level, improving tracking robustness during abrupt directional changes.

The relevance of this work to the proposed Kabaddi system is substantial. Kabaddi involves rapid lunges, dives, and dense tackles where appearance features degrade due to occlusion. The SGM mechanism inspires the identity retention module in AUTOSCORE-KABADDI, while OMC informs the non-rigid motion refinement stage of the tracking pipeline.

2.2 GROUP DETECTION AND GRAPH-BASED SPATIAL MODELING

Understanding group behavior in crowded scenes is essential for modeling defensive formations and collective tackles in Kabaddi.

Lee et al.[2] introduced a real-time human group detection framework integrating YOLOv8 detection with DeepSORT tracking. Their method constructs an adjacency matrix using Euclidean distances and bounding box diagonal ratios to model spatial relationships between individuals. The adjacency matrix is then transformed into a graph structure to analyze dynamic group formations.

Unlike trajectory-only approaches, this framework combines spatial proximity with temporal continuity, enabling robust clustering even under overlapping and dense conditions. The use of graph-based representations for modeling pedestrian relationships directly aligns with the spatial reasoning requirements of Kabaddi, where defenders form chains, blocks, and coordinated holds. The graph construction methodology proposed in this work influences the Dynamic Interaction Graph component of the proposed system. In AUTOSCORE-KABADDI, player proximity and relative positioning are

encoded into interaction proposals, forming the basis for structured reasoning about tackles and raids.

2.3 HUMAN-HUMAN INTERACTION DETECTION IN SPORTS

Automated Kabaddi scoring depends heavily on correctly identifying human-human interactions such as touches, tackles, blocks, and defensive holds.

Wu et al.[3] introduced SportsHHI, a dataset specifically designed for detecting high-level human-human interactions in sports videos . Unlike prior datasets such as VidVRD or AG, which primarily focus on spatial relations or simple object interactions, SportsHHI emphasizes high-level semantic interactions that require detailed spatio-temporal reasoning. The dataset contains 34 interaction classes from basketball and volleyball, annotated across over 11,000 keyframes. Importantly, interactions are formulated as triplets $\langle \text{Subject, Interaction, Object} \rangle$, requiring detection of both participants and classification of their interaction category. The authors propose a two-stage baseline pipeline that first generates exhaustive human pair proposals and then classifies interactions using motion features, contextual cues, relative spatial encoding, and multi-head self-attention mechanisms.

The relevance to Kabaddi is significant. Kabaddi scoring depends on identifying interaction triplets such as $\langle \text{Raider, Touch, Defender} \rangle$ or $\langle \text{Defender, Tackle, Raider} \rangle$. The SportsHHI formulation directly inspires the interaction proposal generation and feature encoding strategy in Module 2 of the proposed system. However, while SportsHHI addresses high-level sports interactions, it does not incorporate explicit rule-based validation aligned with sport-specific scoring logic. This limitation motivates the integration of a dedicated rule reasoning engine in the proposed system.

2.4 FACTOR GRAPH-BASED GROUP BY ACTIVITY

Group-level reasoning is necessary for interpreting complex Kabaddi events such as super tackles, bonus line attempts, and coordinated defensive holds.

Xie et al.[4] proposed the Active Factor Graph Network (AFGN) for group activity recognition. Unlike traditional second-order graph models that only consider pairwise interactions, AFGN introduces third-order factor nodes that model interactions among triplets of individuals. The framework first selects active individuals based on influence scores, pruning irrelevant nodes. It then constructs both a full factor graph and an active factor graph. Factor nodes connect three individuals simultaneously, enabling modeling of higher-order consistency relationships. A consistency-aware reasoning module minimizes an energy function using penalty terms to enforce agreement between individual actions and group-level predictions.

This work directly influences the Interaction Logic Engine in AUTOSCORE-KABADDI. Kabaddi events often involve three or more players simultaneously, such as chain tackles or coordinated blocks. Modeling only pairwise interactions is insufficient for capturing such scenarios. The third-order factor modeling approach provides a structured way to reason about multi-player interactions.

However, AFGN is primarily designed for general group activity recognition and does not integrate domain-specific rule validation. Therefore, while AFGN inspires the factor-based reasoning design, the proposed Kabaddi system extends it with sport-specific scoring logic validation.

2.5 SUMMARY AND RESEARCH GAP

The reviewed literature demonstrates substantial progress in multi-object tracking, interaction modeling, and graph-based reasoning.

Occlusion-robust tracking frameworks introduce embedding reliability management and motion compensation mechanisms to reduce identity switches. Graph-based group detection models effectively represent spatial relationships in crowded environments. SportsHHI establishes a structured benchmark for high-level human-human interaction detection in sports. Active Factor Graph Networks extend interaction modeling to third-order consistency reasoning for group activity recognition.

Despite these advancements, several research gaps remain in the context of Kabaddi. Specifically, existing tracking systems are not tailored for the extreme physical occlusion typical of Kabaddi pile-ups, and current human-human interaction datasets do not model sport-specific rule semantics such as bonus line validation or super tackle conditions. Furthermore, factor graph models lack explicit rule-aware scoring mechanisms aligned with official regulations, and there is currently no unified framework that successfully integrates detection, tracking, interaction reasoning, rule validation, and visualization into a single, automated officiating pipeline.

The proposed AUTOSCORE-KABADDI system addresses these gaps by integrating occlusion-robust tracking, graph-based interaction modeling, third-order factor reasoning, and rule-aware scoring validation into a unified multi-camera vision-based scoring framework tailored specifically for Kabaddi.

2.6 CHALLENGES

The development of AUTOSCORE-KABADDI presents several significant technical challenges that must be addressed to achieve reliable automated scoring in Kabaddi matches.

Kabaddi involves intense physical contact during tackles, chain formations, and pile-ups, resulting in severe and prolonged occlusion. Players are frequently completely obscured for extended periods, making appearance-based re-identification unreliable. Maintaining consistent player identities across occlusion events while avoiding identity switches requires sophisticated embedding management and motion prediction strategies.

Players exhibit highly dynamic and non-linear motion patterns including sudden lunges, dives, directional reversals, and evasive maneuvers. Traditional Kalman filter assumptions of constant velocity fail in such scenarios, necessitating advanced motion compensation techniques that can adapt to abrupt changes in speed and direction.

Kabaddi scoring events often involve simultaneous interactions among three or more players, such as coordinated chain tackles or multiple defenders engaging a single raider. Modeling these higher-order interactions requires moving beyond pairwise relationship modeling to capture complex group dynamics. Distinguishing between incidental proximity and meaningful scoring interactions adds further complexity.

Kabaddi rules involve intricate spatial and temporal constraints, including bonus line validation, lobby entry/exit timing, touch sequencing, and super tackle conditions. Translating these regulations into computational logic requires precise spatial reasoning and temporal event correlation.

CHAPTER 3

SYSTEM ARCHITECTURE

The proposed architecture, as illustrated in **Figure 3.1**, proposes a structured multi-stage pipeline that transforms synchronized multi-camera video streams into refined player trajectories, structured raid-level reasoning, and immersive visualization outputs. The system is divided into three tightly integrated modules: **Module 1** focuses on stable player tracking under occlusion, **Module 2** implements interaction reasoning and scoring logic, and **Module 3** generates visualization and replay outputs.

3.1 MULTI-OBJECT TRACKING IN OCCLUSION

Module 1 ensures that all downstream reasoning operates on stable and reliable positional data. Any error in trajectory estimation would propagate through the system and affect raid interpretation accuracy. Therefore, this module emphasizes robustness against occlusion, abrupt motion, and identity inconsistencies. The module outputs temporally consistent player positions and identities, even during high-density physical interactions such as tackles and defensive formations.

3.1.1 Multi-Camera Synchronization And Acquisition

Video streams are captured from multiple calibrated cameras. Temporal alignment is enforced through timestamp matching and buffering mechanisms. Frame consistency across views is critical for cross-camera triangulation and coordinate transformation. Minor latency differences are compensated using synchronization correction to avoid spatial misalignment.

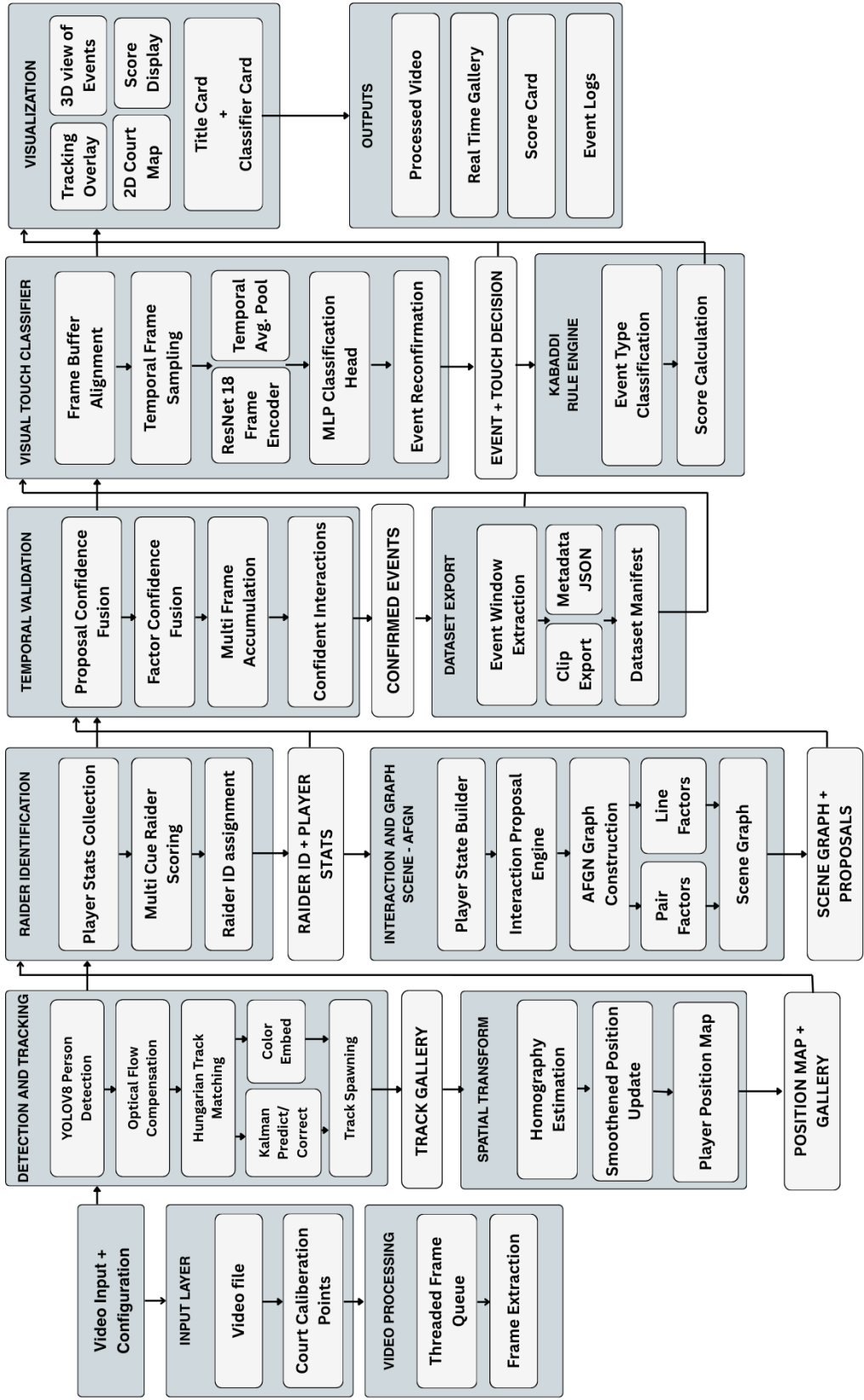


Figure 3.1: Overall System Architecture: Three-Module Pipeline from Multi-Camera Input to Visualization Output

3.1.2 Geometric Calibration And Court Normalization

Each camera undergoes intrinsic calibration to remove lens distortion and extrinsic calibration to estimate camera pose relative to the court. Court markings such as midline, baulk line, and bonus line are detected and used to compute homography matrices. These transformations map image coordinates into a unified court coordinate system. As a result, positional estimates from different cameras become directly comparable in a shared spatial reference.

3.1.3 Player Detection And Tracking Initialization

Deep learning-based object detection models identify player bounding boxes in each frame. A tracking algorithm associates detections across consecutive frames using motion continuity and spatial proximity. This produces initial trajectories representing player movement paths. However, close interactions may cause identity confusion, which requires further correction.

3.1.4 Identity Retention Through Spatial Group Modeling

In kabaddi, defenders often move in coordinated formations. The system leverages this group behavior to maintain consistent player identities. Relative distance matrices between teammates are monitored over time. Sudden changes in identity assignments that violate motion smoothness or team clustering consistency are corrected. This reduces identity switching during dense physical engagement.

3.1.5 Occlusion Handling And PRedictive Motion Estimation

During tackles, players may become partially or fully hidden. The system applies predictive motion estimation using velocity vectors and previous trajectory patterns to infer temporary hidden positions. Context from surrounding players is incorporated to maintain spatial plausibility. When visibility is restored, predicted trajectories are reconciled with new detections to preserve continuity.

3.1.6 Non-Rigid Motion Refinement

Kabaddi players exhibit sharp turns, lunges, and acceleration bursts. Trajectory smoothing is applied using adaptive filters that reduce noise while preserving legitimate abrupt movements. The final output of **Module 1** is a temporally consistent and geometrically accurate trajectory dataset representing all players on the court.

3.2 INTERACTION ANALYSIS AND SCORING LOGIC

Module 2 transforms positional trajectories into structured raid intelligence, as illustrated in **Figure 3.2**. This module is the cognitive core of the system, where spatial relationships are interpreted as meaningful interactions under rule constraints. Rather than relying on simple distance thresholds, it models interactions using dynamic graph structures and probabilistic reasoning. The module outputs comprehensive raid metadata including interaction types, player identities, event sequences, scoring outcomes, and precise timestamps.

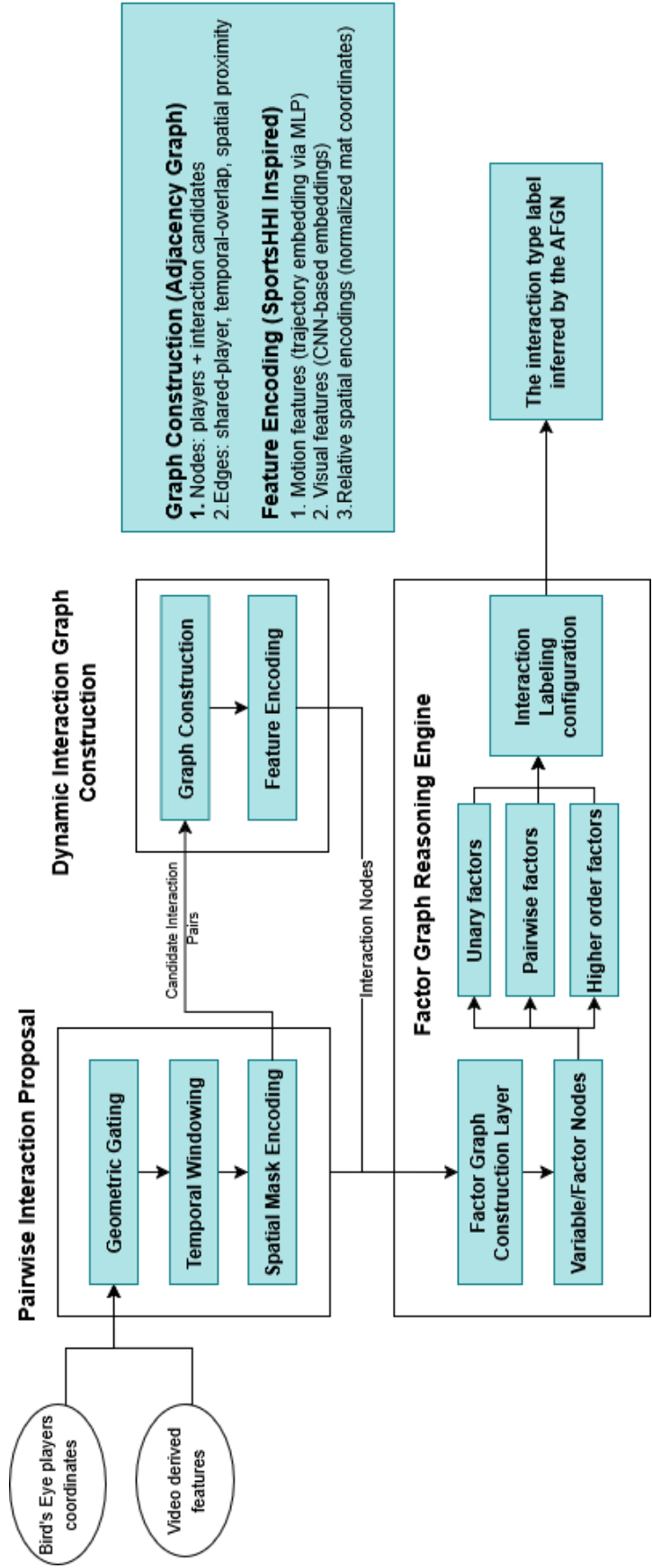


Figure 3.2: Interaction Logic Engine with Graph-Based Reasoning and Rule Validation

3.2.1 Bird's-Eye Coordinate Transformation

Refined trajectories are projected into a top-down coordinate system aligned with court boundaries. This eliminates perspective distortion and simplifies spatial reasoning. Distances between players, boundary crossings, and zone occupancy become directly measurable. Court segmentation enables detection of offensive and defensive regions.

3.2.2 Pairwise Interaction Proposal

The first stage of interaction modeling identifies candidate player pairs that may be interacting. Geometric gating removes pairs whose separation exceeds a defined threshold. Temporal windowing ensures that proximity persists over multiple frames rather than occurring momentarily. Relative velocity direction is analyzed to determine whether players are converging or diverging. Spatial mask encoding captures relative orientation and court zone context. The output is a structured set of candidate interaction edges that represent possible physical engagement.

3.2.3 Active Factor Graph Network Construction

Players are modeled as nodes in a dynamic factor graph that evolves continuously as the raid progresses. Unlike traditional pairwise interaction graphs, the system implements an Active Factor Graph Network (AFGN) that constructs both full and active subgraphs based on influence scoring.

Each player is assigned an influence score computed from interaction proximity, relative velocity, appearance similarity, and spatial relationship to the raider. The top-k most influential players form the active subgraph, reducing

computational complexity while maintaining focus on critical interactions. The raider node is always included in the active set to ensure raid-centric reasoning.

The graph structure captures multi-player tackles through third-order factor nodes that model interactions among triplets of players. This higher-order modeling enables the system to reason about coordinated defensive formations, containment strategies, and simultaneous multi-defender engagement patterns that cannot be captured by pairwise edges alone.

3.2.4 Multi-Level Factor Node Construction

The AFGN constructs three types of factor nodes to model interactions at different granularities. First-order pair factors connect two players and encode pairwise interaction features including Euclidean distance, relative velocity, approach score, adjacency ratio, and track confidence. These factors are classified as `RAIDER_DEFENDER_PAIR` or `DEFENDER_RAIDER_PAIR` based on participant roles.

Second-order line factors model player-boundary interactions, connecting individual players to critical court lines such as baulk line, bonus line, and end line. These factors encode distance to line, crossing status, and track confidence, enabling detection of boundary violations and bonus attempts.

Third-order factor nodes model coordinated multi-player interactions through triplet structures. `THIRD_ORDER_PRESSURE` factors capture defensive compactness by measuring inter-defender distances and spatial spread. `DEFENDER_CONTAINMENT` factors evaluate enclosure angles to detect when multiple defenders surround the raider. These higher-order factors enable reasoning about group tactics that cannot be inferred from pairwise relationships alone.

3.2.5 Graph-Based Pressure Scoring And Inference

The AFGN implements graph-based pressure scoring to quantify defensive intensity and raider containment. For `THIRD_ORDER_PRESSURE` factors, the system computes compactness scores based on mean inter-player distances and spread metrics. For `DEFENDER_CONTAINMENT` factors, enclosure angles and spatial spread are combined to measure containment strength.

Global context features are extracted from the full factor graph, including best contact score, best containment score, defensive depth variance, and width spread. These global features provide scene-level understanding that complements local interaction reasoning.

The system maintains both a full factor graph containing all visible players and an active factor graph containing only the top-k influential nodes. This dual-graph architecture enables comprehensive scene understanding while focusing computational resources on the most relevant interactions. Influence-based node selection ensures that critical players involved in scoring events are always included in the active reasoning set.

3.2.6 Temporal Interaction Validation And Confidence Fusion

Interaction proposals are validated across temporal windows to filter spurious detections and ensure consistent evidence. The Temporal Interaction Candidate Manager maintains active candidates for each unique interaction key, tracking frame sequences, confidence scores, and factor confidences.

Confirmation requires minimum frame count thresholds and fused confidence scores that combine proposal confidence and factor graph

confidence. For raider-defender contact events, the system requires at least 2 frames with fused confidence exceeding 0.58. For line crossing events, single-frame confirmation is permitted with higher confidence thresholds.

Each confirmed event is annotated with temporal window boundaries including window start, window end, core window start, and core window end. These temporal annotations enable extraction of interaction clips for downstream visual confirmation and classifier training. The system maintains pre-context and post-context buffers to capture interaction dynamics before and after the peak interaction moment.

3.2.7 Kabaddi-Specific Reasoning Engine

The Kabaddi AFGN Reasoning Engine processes scene graphs and confirmed events to infer scoring actions according to official kabaddi rules. The engine builds a comprehensive context structure containing raider state, defender positions, pairwise contact factors, line crossing factors, third-order pressure factors, and global scene context.

Four categories of events are inferred: raid progress events (lobby entry, baulk crossing, bonus line crossing, midline return), contact events (raider-defender contact, multiple defender touch, lobby activation), defender events (tackle, assist tackle, super tackle trigger), and boundary events (end line crossing, out of bounds).

Rule-based validation applies consistency scoring across multiple dimensions. Contact legality checks verify baulk line crossing before awarding touch points. Bonus legality validates defender count and line crossing direction. Tackle legality ensures raider speed reduction and sufficient defensive pressure. Return legality confirms midline crossing after successful raid

completion. These consistency scores modulate action confidence to ensure rule compliance.

3.2.8 Visual Confirmation Through Touch Classification

Certain interaction events, particularly raider-defender contact, require visual confirmation beyond geometric proximity and temporal consistency. The system integrates a learned touch classifier to validate contact events using visual evidence from interaction windows.

When a raider-defender contact event is confirmed through temporal validation, the system extracts a temporal window of frames centered on the interaction. This window is passed to a ResNet18-based touch classifier that performs frame-level feature extraction followed by temporal average pooling. The classifier outputs binary predictions: `valid_touch` or `no_touch`, with associated confidence scores. The touch classifier is trained on labeled interaction windows exported during system operation. The Confirmed Window Dataset Exporter automatically saves interaction clips along with metadata including event type, temporal boundaries, subject and object player IDs, and factor confidence scores. These exported windows are manually labeled and used to train the classifier, creating a feedback loop that continuously improves visual confirmation accuracy.

The Classifier Bridge integrates both heuristic scoring and learned model predictions. When a trained model checkpoint is available, the bridge uses the touch classifier for raider-defender contact events. For other event types or when no model is available, the bridge falls back to heuristic confidence scoring. This hybrid approach ensures robust operation while enabling gradual improvement through supervised learning.

3.2.9 Comprehensive Raid Metadata and Event Logging

After completing interaction reasoning, rule validation, and visual confirmation, the system generates comprehensive raid metadata that documents all scoring events with full traceability. Each confirmed event includes event type, event family classification, frame index, temporal window boundaries, subject and object player IDs, line name for boundary interactions, proposal confidence, factor confidence, and visual confirmation results.

The system maintains a confirmed event log that accumulates all validated scoring actions throughout the match. Deduplication mechanisms prevent duplicate point awards for the same interaction. Touched defender sets track which defenders have been contacted during the current raid to ensure each defender contributes at most one touch point per raid.

Event metadata is exported in structured JSON format containing event details, temporal annotations, confidence scores, classifier predictions, and scene context. This structured output enables post-match analysis, referee review, and continuous system improvement through supervised learning on exported interaction windows.

3.3 VISUALIZATION AND REPLAY RECONSTRUCTION

Module 3 focuses on converting structured raid metadata into clear, intuitive, and visually meaningful outputs. While the previous modules concentrate on detection and reasoning, this module is responsible for presentation and interpretation. Its main goal is to ensure that complex computational results are transformed into visuals that can be easily understood by referees, coaches, analysts, and viewers.

Rather than presenting raw numbers or abstract interaction labels, **Module 3** creates a visual story of the raid. It reconstructs player movements, highlights important interactions, updates the scoreboard automatically, and provides replay options from different viewing angles. The factor graph structure is visualized through interactive graph panels showing active nodes, pair factors, line factors, and third-order factor nodes. This helps users understand not only what decision was made, but also why that decision was made. By combining spatial reconstruction, graph visualization, and synchronized scoring updates, this module enhances trust in automated analysis systems.

3.3.1 3D Coordinate Reprojection

The first step in visualization is converting bird's-eye player coordinates into three-dimensional court coordinates. The system estimates vertical positioning using court geometry and camera calibration parameters. This allows the system to generate a 3D model of the court where players are placed at realistic positions. The transformation from 2D coordinates to 3D space ensures that reconstructed scenes reflect actual spatial relationships between players.

This reprojection process enables realistic replay rendering. Movements such as diving tackles, leaning motions, and group formations become visually clear in the reconstructed environment. As a result, users can better understand the physical dynamics of the raid.

3.3.2 Interactive Graph Visualization and Decision Dashboard

Module 3 includes an interactive graph visualization panel that renders the Active Factor Graph Network structure in real time. The graph

panel displays active nodes as colored circles (red for raider, blue for defenders) positioned according to court coordinates. Pair factors are shown as connecting edges with thickness proportional to interaction strength. Third-order factor nodes are rendered as diamond shapes positioned at the centroid of participant triplets, with connecting lines to involved players.

The visualization uses color coding to distinguish factor types: `THIRD_ORDER_PRESSURE` factors appear in orange, `DEFENDER_CONTAINMENT` factors in purple. Node sizes reflect track confidence, and velocity vectors indicate player motion direction and speed. This graph visualization provides intuitive insight into the reasoning process, showing which players are considered active, which interactions are being evaluated, and how higher-order factors contribute to scoring decisions.

The interactive decision dashboard displays structured raid summaries along with visual highlights of important events. Users can navigate through the timeline of the raid and inspect specific frames where interactions occurred. The dashboard provides clear explanations of scoring decisions, including which rule was applied, what factor confidence was computed, and whether visual confirmation was obtained. This transparency improves trust in automated decision systems. The interface includes zoom controls, replay speed adjustment, and player highlighting features. This makes analysis more efficient and easier to follow.

3.3.3 Dynamic Scoreboard Overlay Generation

One of the key outputs of **Module 3** is automatic scoreboard overlay generation. Based on validated scoring metadata, the system updates the scoreboard in real time. The overlay displays raid number, team scores, bonus confirmations, and successful tackles. These updates are synchronized with

replay footage so that scoring changes appear at the correct moment in the video. This synchronization ensures consistency between visual replay and scoring information.

The scoreboard design follows broadcast standards to maintain clarity. By automating this process, the system reduces manual intervention and minimizes human error in score reporting.

3.3.4 Immersive 3D Replay Rendering

The final component of **Module 3** is immersive 3D replay rendering. The reconstructed 3D environment allows users to view the raid from multiple angles. Instead of relying on fixed camera footage, the system can generate custom viewpoints such as side view, top view, or close-up interaction view.

The integration of these modules creates a reliable and scalable system capable of supporting real-time analysis and post-match review. By combining technical accuracy with clear visualization, the system enhances transparency, improves officiating support, and increases viewer engagement.

The Active Factor Graph Network enables higher-order reasoning about multi-player interactions, while the touch classifier provides visual confirmation of contact events. The dataset export pipeline creates a continuous improvement loop where system outputs are used to train better classifiers. This architecture demonstrates how advanced computer vision, graph-based reasoning, and supervised learning techniques can be effectively integrated to create a practical and interpretable sports analytics system.

CHAPTER 4

IMPLEMENTATION

This chapter describes the practical realization of the AUTOSCORE-KABADDI system, covering multi-object tracking, Active Factor Graph Network construction, interaction reasoning, visual confirmation through learned classifiers, and visualization.

4.1 MULTI-OBJECT TRACKING UNDER OCCLUSION

This module implements robust player tracking under extreme occlusion conditions typical in kabaddi matches. The tracking system maintains consistent player identities throughout the match by combining deep learning-based detection, Kalman filtering for motion prediction, optical flow for motion compensation, and appearance-based re-identification.

4.1.1 Yolov8-Based Player Detection

The detection module identifies player bounding boxes in each frame using the YOLOv8n model, which provides an optimal balance between speed and accuracy for real-time processing. Each detection is augmented with appearance features extracted from HSV color histograms.

$$e_{norm} = \frac{h}{\|h\|_2} \quad (4.1)$$

where h is the raw HSV histogram ($8 \times 8 \times 8$ bins) and e_{norm} is the normalized 512-dimensional embedding vector.

Algorithm 1 YOLOv8 Player Detection and Embedding Extraction

- 1: **Input:** Video frame I , YOLO model M , Confidence threshold θ_{conf}
 - 2: **Output:** Detection set D with bounding boxes and appearance embeddings
 - 3: Execute YOLOv8 inference on frame I using GPU acceleration
 - 4: Filter detections to retain only person class (class ID = 0) with confidence $\geq \theta_{conf}$
 - 5: **for** each valid detection **do**
 - 6: Extract bounding box coordinates (x_1, y_1, x_2, y_2)
 - 7: Compute foot position as bottom-center point: $foot = (\frac{x_1+x_2}{2}, y_2)$
 - 8: Extract appearance embedding from cropped player region using HSV color histogram
 - 9: Normalize histogram using L2 normalization (Equation (4.1))
 - 10: Store detection with bounding box, foot position, embedding, and confidence score
 - 11: **end for**
 - 12: **return** Detection set D containing all valid player detections
-

Algorithm 1 performs player detection and appearance feature extraction in a single pass. The YOLOv8n model runs on GPU, achieving inference times of 10-20ms per frame. Each detection is filtered by confidence threshold (default 0.4) to reduce false positives. The foot position serves as the primary spatial reference for tracking, as it remains stable across different player poses. Appearance embeddings are computed using 8×8×8 bin HSV color histograms, providing a 512-dimensional feature vector that captures jersey color and appearance characteristics while remaining robust to lighting variations and pose changes.

4.1.2 Kalman Filter Motion Prediction With Optical Flow Compensation

Each tracked player is associated with a Kalman filter that predicts future positions based on a constant velocity motion model. To handle camera motion and rapid non-rigid movements, the system augments Kalman prediction with Lucas-Kanade optical flow tracking.

Algorithm 2 Kalman Filter with Optical Flow Motion Compensation

- 1: **Input:** Previous frame I_{t-1} , Current frame I_t , Player gallery $\mathcal{G}$
 - 2: **Output:** Updated Kalman states with motion compensation
 - 3: **for** each player p in gallery $\mathcal{G}$ **do**
 - 4: Predict next position using Kalman filter
 - 5: Apply state transition: $\mathbf{x}_{t|t-1} = \mathbf{F}\mathbf{x}_{t-1}$ where $\mathbf{F}$ is defined in Equation (4.2)
 - 6: Update error covariance: $\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^T + \mathbf{Q}$
 - 7: **if** player has optical flow tracking points **then**
 - 8: Compute Lucas–Kanade optical flow between frames
 - 9: Calculate mean displacement $(\Delta x, \Delta y)$
 - 10: Apply motion compensation: $\mathbf{x}_{t|t-1}[0:2] \leftarrow \mathbf{x}_{t|t-1}[0:2] + (\Delta x, \Delta y)$
 - 11: Update flow tracking points
 - 12: **end if**
 - 13: **end for**
 - 14: **return** Motion-compensated Kalman predictions
-

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (4.2)$$

Algorithm 2 combines Kalman filtering with optical flow to achieve robust motion prediction under camera movement. The Kalman filter maintains a 4-dimensional state vector $[x, y, v_x, v_y]^T$ representing position and velocity. The transition matrix $\mathbf{F}$ in **Equation (4.2)** implements a constant velocity model with process noise covariance $\mathbf{Q} = 0.03 \cdot \mathbf{I}_4$ and measurement noise $\mathbf{R} = 0.5 \cdot \mathbf{I}_2$. Optical flow tracking uses a 15×15 pixel window with 3-level pyramid decomposition to track feature points extracted from each player’s bounding box. When sufficient flow points are successfully tracked (minimum 5 points), their mean displacement compensates the Kalman prediction, effectively handling camera jitter and panning. This hybrid approach combines the smoothness of Kalman filtering with the adaptability of optical flow, resulting in stable tracking even during rapid camera movements.

4.1.3 Hungarian Algorithm For Tracking-Detection Association

The system uses the Hungarian algorithm to solve the optimal assignment problem between predicted tracks and new detections, employing a multi-component cost function that balances spatial proximity, appearance similarity, and geometric consistency.

Algorithm 3 Track-Detection Association using Hungarian Algorithm

- 1: **Input:** Player gallery $\mathcal{G}$, Detections D , Cost threshold θ_{cost}
 - 2: **Output:** Matched pairs M , Unmatched tracks U_t , Unmatched detections U_d
 - 3: Generate Kalman predictions for all active tracks in gallery $\mathcal{G}$
 - 4: Initialize cost matrix $C \in \mathbb{R}^{|\mathcal{G}| \times |D|}$
 - 5: **for** each track i in $\mathcal{G}$ **do**
 - 6: **for** each detection j in D **do**
 - 7: Compute cost components as defined in Equation (4.3)
 - 8: Compute weighted total cost using Equation (4.4)
 - 9: Store result in $C[i, j]$
 - 10: **end for**
 - 11: **end for**
 - 12: Apply Hungarian algorithm to find optimal assignment minimizing total cost
 - 13: Retain only matches where $C[i, j] < \theta_{cost}$
 - 14: Identify unmatched tracks and unmatched detections
 - 15: **return** Matched pairs M , unmatched tracks U_t , unmatched detections U_d
-

$$\begin{aligned}
 c_{spatial} &= \frac{\|\text{pred}_i - \text{foot}_j\|_2}{220} \\
 c_{appear} &= 1 - \text{cosine}(\text{emb}_i, \text{emb}_j) \\
 c_{anchor} &= 1 - \text{cosine}(\text{anchor}_i, \text{emb}_j) \\
 c_{size} &= \left| \frac{h_i - h_j}{h_i} \right|
 \end{aligned} \tag{4.3}$$

$$C[i, j] = w_{spatial} \cdot c_{spatial} + w_{appear} \cdot c_{appear} + w_{anchor} \cdot c_{anchor} + w_{size} \cdot c_{size} \tag{4.4}$$

where adaptive weights are: $w_{spatial} = 0.4$, $w_{appear} = 0.3$, $w_{anchor} = 0.2$, $w_{size} = 0.1$ for mature tracks.

Algorithm 3 solves the bipartite matching problem between existing tracks and new detections using the Hungarian algorithm with $O(n^3)$ complexity. The cost function in **Equation (4.4)** combines four complementary cues to achieve robust association under occlusion and appearance changes. Spatial cost measures normalized Euclidean distance between predicted and detected foot positions, with normalization factor 220 pixels representing typical court width.

Appearance cost compares current embeddings using cosine similarity, while anchor cost compares against the initial appearance captured when the track was first created, providing robustness to temporary appearance changes. Size cost enforces height consistency, preventing association between players of significantly different sizes. The cost threshold $\theta_{cost} = 0.65$ filters out unreliable matches, allowing unmatched detections to initialize new tracks.

4.1.4 Score-Based Gallery Management

The gallery maintains player identities using a scoring mechanism that balances appearance consistency and temporal continuity, handling temporary occlusions and long-term track maintenance.

$$emb_{new} = (1 - \alpha) \cdot emb_{old} + \alpha \cdot emb_{detection} \quad (4.5)$$

where $\alpha = 0.25$ is the smoothing factor that balances stability and adaptability.

Algorithm 4 Gallery Update and Track Management

```

1: Input: Matched pairs  $M$ , Detections  $D$ , Gallery  $\mathcal{G}$ , Unmatched tracks  $U_t$ ,
   Unmatched detections  $U_d$ 
2: Output: Updated gallery  $\mathcal{G}'$  with refined player tracks
3: for each matched pair  $(i, j) \in M$  do
4:   Reset miss counter:  $\mathcal{G}[i].age \leftarrow 0$ 
5:   Update Kalman filter with detection:  $\mathcal{G}[i].\mathcal{K}.update(D[j].foot)$ 
6:   Smooth appearance embedding using Equation (4.5)
7:   Update bounding box and confidence score
8: end for
9: for each unmatched track  $i \in U_t$  do
10:  Increment miss counter:  $\mathcal{G}[i].age \leftarrow \mathcal{G}[i].age + 1$ 
11:  if  $\mathcal{G}[i].age > \text{MAX\_AGE}$  then
12:    Remove track from gallery
13:  end if
14: end for
15: for each unmatched detection  $j \in U_d$  do
16:  Initialize new track with Kalman filter at  $D[j].foot$ 
17:  Set appearance anchor to  $D[j].emb$ 
18:  Add new track to gallery with initial confidence
19: end for
20: return Updated gallery  $\mathcal{G}'$ 

```

Algorithm 4 maintains the player gallery through matched updates, occlusion handling, and new track initialization. For matched tracks, the Kalman filter is updated with the new measurement, and the appearance embedding is smoothed using exponential moving average (**Equation** (4.5)) to gradually adapt to appearance changes while maintaining consistency. Unmatched tracks increment their age counter, and tracks exceeding $\text{MAX_AGE} = 200$ frames (approximately 6.7 seconds at 30 FPS) are removed to prevent gallery bloat. New detections that cannot be matched to existing tracks initialize fresh tracks, allowing the system to handle players entering the frame or recovering from prolonged occlusions. The smoothing factor $\alpha = 0.25$ ensures that appearance updates are gradual, preventing sudden identity switches due to temporary appearance variations.

4.1.5 Homography-Based Court Normalization

The system transforms image coordinates to a standardized bird's-eye court coordinate system using homography mapping, enabling spatial reasoning for interaction detection and tactical analysis.

Algorithm 5 Court Homography Computation

- 1: **Input:** Court line definitions $L = \{l_{baulk}, l_{bonus}, l_{middle}, l_{end}\}$
 - 2: **Output:** Homography matrix $H \in \mathbb{R}^{3 \times 3}$
 - 3: Define court lines using endpoints in image space
 - 4: **for** each line $l \in L$ **do**
 - 5: Compute line equation $ax + by + c = 0$
 - 6: **end for**
 - 7: Compute intersection points of the court boundaries
 - 8: Define correspondences between image points P_{img} and court points P_{court} in meters
 - 9: Estimate homography using RANSAC [?] based on Equation (4.6)
 - 10: **return** Homography matrix H
-

$$\begin{bmatrix} x_{court} \\ y_{court} \\ 1 \end{bmatrix} = H \begin{bmatrix} x_{img} \\ y_{img} \\ 1 \end{bmatrix} \quad (4.6)$$

where H is the 3×3 homography matrix mapping image coordinates to court coordinates (10m $\times$ 6.5m standard kabaddi court).

Algorithm 5 computes the perspective transformation between image space and a standardized court coordinate system. Court lines (baulk line at 3.75m, bonus line at 4.75m, end line at 6.5m) are manually annotated in the first frame, and their intersections define correspondence points. The homography matrix H in **Equation** (4.6) is estimated using RANSAC to handle annotation noise. Once computed, this matrix transforms all player foot positions from pixel coordinates to metric court coordinates, enabling distance-based interaction detection and tactical positioning analysis. The

bird's-eye view representation normalizes perspective distortion, making spatial relationships consistent regardless of camera angle.

This comprehensive tracking module forms the foundation for all subsequent analytical operations. By maintaining consistent player identities under severe occlusion, the system enables reliable interaction detection and scoring analysis in the downstream modules.

4.2 ACTIVE FACTOR GRAPH NETWORK AND INTERACTION REASONING

This module implements structured reasoning over player interactions using Active Factor Graph Networks (AFGN) to detect and validate scoring events according to official kabaddi rules. The system generates interaction proposals based on spatial proximity and temporal consistency, constructs dynamic factor graphs with third-order modeling, performs temporal validation with confidence fusion, and applies rule-based validation to determine point assignments. Visual confirmation is provided through a learned touch classifier for contact events.

4.2.1 Interaction Proposal Generation

The system generates candidate interaction proposals based on spatial proximity and temporal consistency, distinguishing between human-human interactions (HHI) and human-line interactions (HLI).

$$\begin{aligned}
 rel_vel &= (vel_r - vel_d) \cdot \frac{pos_d - pos_r}{\|pos_d - pos_r\|} \\
 angle &= \arctan 2(pos_d.y - pos_r.y, pos_d.x - pos_r.x)
 \end{aligned} \tag{4.7}$$

Algorithm 6 Pairwise Interaction Proposal Generation

```

1: Input: Player states  $S$ , Raider ID  $r$ , Distance threshold  $\theta_{dist}$ 
2: Output: Interaction proposals  $P = \{p_1, p_2, \dots, p_k\}$ 
3: Initialize empty proposal set  $P \leftarrow \emptyset$ 
4: for each defender  $d$  in  $S \setminus \{r\}$  do
5:   Compute Euclidean distance:  $dist = \|pos_r - pos_d\|_2$ 
6:   if  $dist < \theta_{dist}$  then
7:     Create HHI proposal with features from Equation (4.7)
8:   end if
9: end for
10: for each critical court line (baulk, bonus, end) do
11:   Check if raider crossed line using current and previous positions
12:   if crossing detected then
13:     Create HLI proposal with direction and distance
14:   end if
15: end for
16: return Proposal set  $P$  containing all detected interactions

```

Algorithm 6 generates structured interaction proposals representing potential scoring events. For human-human interactions, the algorithm computes pairwise distances between the raider and all defenders, creating proposals when distance falls below $\theta_{dist} = 2.5$ meters. Each HHI proposal includes relative velocity and approach angle to capture interaction dynamics. For human-line interactions, the algorithm detects line crossings by comparing current and previous raider positions against critical court lines (baulk at 3.75m, bonus at 4.75m, end at 6.5m). Line crossing direction (forward/backward) is recorded to distinguish between raid progression and retreat. All proposals are timestamped and tagged with interaction type for downstream validation.

4.2.2 Automatic Raider Identification

The system automatically identifies the raider based on movement patterns and court position analysis, eliminating the need for manual annotation.

Algorithm 7 Automatic Raider Identification

- 1: **Input:** Player gallery $\mathcal{G}$, Raider statistics R , Court lines
 - 2: **Output:** Raider ID r or None
 - 3: Filter candidates: select players visible for at least 20 frames
 - 4: **for** each candidate p **do**
 - 5: Compute raider score using Equation (4.8)
 - 6: **end for**
 - 7: Select player with highest score exceeding minimum threshold
 - 8: Validate that selected player crossed baulk line (minimum y-coordinate < 3.75 m)
 - 9: **return** Raider ID r or None if insufficient evidence
-

$$score = w_1 \cdot |v_y| + w_2 \cdot \max(0, y_{bonus} - y_{min}) + w_3 \cdot d_{isolation} \quad (4.8)$$

where $w_1 = 0.4$ (motion weight), $w_2 = 0.4$ (penetration weight), $w_3 = 0.2$ (isolation weight).

Algorithm 7 identifies the raider by analyzing movement patterns and spatial positioning. The raider score in **Equation** (4.8) combines three complementary cues: forward velocity magnitude $|v_y|$ indicating aggressive court penetration, penetration depth beyond the bonus line showing raid commitment, and spatial isolation $d_{isolation}$ measuring distance from other players. The algorithm requires a minimum observation window of 20 frames to ensure statistical reliability and validates that the candidate actually crossed the baulk line to eliminate false positives. Weights are tuned to prioritize motion and penetration over isolation, as raiders may cluster with defenders during tackles. The identification process runs every 30 frames (1 second) to adapt to raid transitions.

4.2.3 Temporal Interaction Validation

Interaction candidates are validated over temporal windows to filter spurious detections and ensure consistent evidence across multiple frames.

Algorithm 8 Temporal Interaction Validation

```

1: Input: Frame proposals  $P_t$ , Candidate buffer  $C$ , Window size  $w$ 
2: Output: Confirmed events  $E_{confirmed}$ 
3: for each proposal  $p \in P_t$  do
4:   Create unique key ( $type, subject, object$ )
5:   Update candidate buffer: add current frame to existing candidate or create
     new entry
6: end for
7: for each candidate  $c \in C$  do
8:   Compute temporal density:  $density = \frac{|frames|}{duration}$ 
9:   if  $|frames| \geq 3$  and  $density > 0.5$  then
10:    Confirm event using Equation (4.9)
11:   end if
12: end for
13: Remove stale candidates older than window size  $w$ 
14: return Confirmed events  $E_{confirmed}$  with confidence scores

```

$$confidence = \min \left(1.0, \frac{|frames|}{10} \right) \quad (4.9)$$

Algorithm 8 filters spurious interaction detections by requiring consistent evidence across temporal windows. Each interaction candidate is tracked across frames, and confirmation requires at least 3 detections with temporal density exceeding 50%. Temporal density prevents confirmation of interactions that appear sporadically, ensuring that only sustained interactions are validated. Event confidence **Equation** (4.9) increases with the number of supporting frames, saturating at 10 frames to provide a normalized confidence metric. The window size $w = 30$ frames (1 second) balances responsiveness with noise rejection. Stale candidates are pruned to prevent memory bloat and ensure the buffer contains only recent interaction evidence.

4.2.4 Active Factor Graph Network Construction

The system constructs an Active Factor Graph Network (AFGN) that models player interactions through multi-level factor nodes, enabling higher-order reasoning about coordinated multi-player events.

Algorithm 9 Active Factor Graph Network Construction

- 1: **Input:** Proposals P , Gallery $\mathcal{G}$, Raider ID r , Top-k parameter k
 - 2: **Output:** Active factor graph G_{active} , Full factor graph G_{full}
 - 3: Initialize influence scores for all visible players
 - 4: **for** each HHI proposal **do**
 - 5: Compute influence contribution using Equation (4.10)
 - 6: **end for**
 - 7: **for** each player near raider **do**
 - 8: Add proximity-based influence
 - 9: **end for**
 - 10: **for** each HLI proposal **do**
 - 11: Add line-based influence bonus
 - 12: **end for**
 - 13: Sort players by influence score and select top-k as active nodes
 - 14: Build adjacency matrices for both full and active node sets
 - 15: Construct pair factors, line factors, and third-order factor nodes
 - 16: Compute global context features from full graph
 - 17: **return** Active graph G_{active} and full graph G_{full}
-

$$w_{dist} = \frac{1}{d_{ij} + \epsilon}, \quad w_{vel} = \max(0.25, v_{rel})$$

$$w_{feat} = \max(0.1, \cosine(e_i, e_j) + 0.25) \quad (4.10)$$

$$\text{influence}_i += \alpha_s \cdot w_{dist} \cdot w_{vel} \cdot w_{feat}$$

where $\alpha_s = 1.35$ if player i is raider, $\alpha_s = 1.2$ if player j is raider, else $\alpha_s = 1.0$.

Algorithm 9 implements influence-based node selection to construct both full and active factor graphs. Influence scoring combines distance-based weights, relative velocity, and appearance feature similarity to identify the

most relevant players for interaction reasoning. The raider receives a fixed influence boost of 5.0 to ensure inclusion in the active set. Defenders near the raider receive additional influence based on proximity and closing speed. Line interaction proposals contribute influence bonuses for raider-line interactions (1.25 for baulk/bonus lines) and boundary proximity (0.45 for end line). The top-k players by influence form the active node set, reducing computational complexity while maintaining focus on critical interactions. Adjacency matrices encode size ratios between player bounding boxes to capture relative scale information.

4.2.5 Third-Order Factor Node Construction

The AFGN constructs third-order factor nodes to model coordinated multi-player interactions that cannot be captured by pairwise edges alone.

Algorithm 10 Third-Order Factor Node Construction

```

1: Input: Active nodes  $N$ , Gallery  $\mathcal{G}$ , Raider ID  $r$ 
2: Output: Third-order factor nodes  $F_{3rd}$ 
3: Initialize empty factor set  $F_{3rd} \leftarrow \emptyset$ 
4: Identify defenders  $D = N \setminus \{r\}$ 
5: for each defender pair  $(d_1, d_2)$  where  $d_1 \neq d_2$  do
6:   Compute distances:  $d_{r,d_1}, d_{r,d_2}, d_{d_1,d_2}$ 
7:   Compute spread:  $s = \max(d_{r,d_1}, d_{r,d_2}, d_{d_1,d_2})$ 
8:   Compute compactness:  $c = \max(0, 1 - \frac{\text{mean}(d_{r,d_1}, d_{r,d_2}, d_{d_1,d_2})}{1.8})$ 
9:   if  $c \geq 0.15$  then
10:    Create THIRD_ORDER_PRESSURE factor
11:   end if
12:   Compute enclosure angle  $\theta$  using Equation (4.11)
13:   if  $\theta > 0.35$  then
14:    Create DEFENDER_CONTAINMENT factor
15:   end if
16: end for
17: return Third-order factor nodes  $F_{3rd}$ 

```

$$\theta = \frac{\arccos\left(\frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\| \|\vec{v}_2\|}\right)}{\pi} \quad (4.11)$$

where $\vec{v}_1 = \text{pos}_{d_1} - \text{pos}_r$ and $\vec{v}_2 = \text{pos}_{d_2} - \text{pos}_r$.

Algorithm 10 constructs third-order factor nodes that model triplet interactions among the raider and defender pairs. Third order pressure factors capture defensive compactness by measuring inter-player distances and computing a compactness score that increases when all three players are close together. The spread metric identifies the maximum pairwise distance, while compactness normalizes the mean distance by a threshold of 1.8 meters. Defender containment factors evaluate enclosure angles to detect when two defenders position themselves on opposite sides of the raider, creating a containment formation. The enclosure angle is computed as the normalized angle between vectors from raider to each defender, with values exceeding 0.35 (approximately 63 degrees) indicating significant containment. These third-order factors enable the system to reason about coordinated defensive tactics such as pincer movements, surrounding formations, and multi-defender tackles.

4.2.6 Kabaddi AFGN Reasoning and Scoring

The Kabaddi AFGN Reasoning Engine processes scene graphs and confirmed events to infer scoring actions according to official kabaddi rules.

$$\begin{aligned}
 c_{tackle} = & 0.25 \cdot p_{higher_order} + 0.20 \cdot p_{graph} \\
 & + 0.25 \cdot \max(0, 1 - v_{raider}/0.35) \\
 & + 0.20 \cdot \min(1, n_{nearby}/3) \\
 & + 0.10 \cdot c_{containment}
 \end{aligned} \tag{4.12}$$

where p_{higher_order} is higher-order pressure, p_{graph} is graph pressure score, v_{raider} is raider speed, n_{nearby} is nearby defender count, and $c_{containment}$ is best containment score.

Algorithm 11 Kabaddi AFGN Reasoning and Scoring

- 1: **Input:** Scene graph G , Confirmed events E , Raider ID r , Frame index t
 - 2: **Output:** Scoring actions A with point assignments
 - 3: Build context structure containing raider state, defenders, pair factors, line factors, third-order factors, and global context
 - 4: Infer raid progress events: lobby entry, baulk crossing, bonus line crossing, midline return
 - 5: Infer contact events using pairwise contact factors and graph pressure scores
 - 6: Infer defender events: tackle, assist tackle, super tackle using Equation (4.12)
 - 7: Infer boundary events: end line crossing, out of bounds
 - 8: Apply consistency scoring: contact legality, bonus legality, tackle legality, return legality
 - 9: Deduplicate actions and filter by confidence thresholds
 - 10: **return** Scoring actions A with points, team, confidence, and metadata
-

Algorithm 11 implements the Kabaddi-specific reasoning engine that converts scene graphs into scoring actions. The context structure aggregates all relevant information including raider position and velocity, defender positions, pairwise contact factors, line crossing factors, third-order pressure factors, containment factors, and global scene context. Four categories of events are inferred: raid progress events track raider movement through court zones, contact events detect raider-defender touches, defender events identify tackles and assists, and boundary events detect violations. Tackle confidence **Equation** (4.12) combines higher-order pressure from third-order factors, graph pressure scores, raider speed reduction, nearby defender support, and containment scores. Consistency scoring validates actions against kabaddi rules: contact legality requires baulk crossing, bonus legality validates defender count and line crossing direction, tackle legality ensures sufficient pressure and speed reduction, and return legality confirms midline crossing. Deduplication prevents multiple point awards for the same interaction by maintaining emitted action keys per raid.

4.2.7 Visual Confirmation Through Touch Classification

Certain interaction events require visual confirmation beyond geometric proximity and temporal consistency. The system integrates a learned touch classifier to validate contact events using visual evidence.

Algorithm 12 Touch Classifier Model Architecture and Inference using ResNet18

- 1: **Input:** Interaction window frames $F = \{f_1, \dots, f_T\}$, Classifier model M
 - 2: **Output:** Touch prediction $\{p_{no_touch}, p_{valid_touch}\}$
 - 3: Resize each frame to 224×224 pixels and normalize to [0, 1]
 - 4: **for** each frame f_t **do**
 - 5: Extract features using ResNet18 backbone: $h_t = \text{ResNet18}(f_t)$
 - 6: **end for**
 - 7: Aggregate temporal features using average pooling: $h_{pooled} = \frac{1}{T} \sum_{t=1}^T h_t$
 - 8: Apply classification head with dropout and ReLU: $z = \text{Head}(h_{pooled})$
 - 9: Compute softmax probabilities: $p = \text{softmax}(z)$
 - 10: **return** Touch probabilities $\{p_{no_touch}, p_{valid_touch}\}$
-

Algorithm 12 implements a ResNet18-based touch classifier with temporal average pooling for contact validation. The model processes a temporal window of $T = 12$ frames centered on the interaction event. Each frame is independently encoded through a ResNet18 backbone pretrained on ImageNet, producing a 512-dimensional feature vector. Temporal aggregation uses average pooling to combine frame-level features into a single representation that captures the interaction dynamics across the window. The classification head consists of dropout (0.2), linear layer (512→128), ReLU activation, dropout (0.2), and final linear layer (128→2) producing logits for no_touch and valid_touch classes. Softmax normalization converts logits to probabilities. The classifier is trained using cross-entropy loss on labeled interaction windows exported during system operation.

4.3 CONFIRMED EVENT ARCHIVAL AND DATASET GENERATION

This module implements automated export of confirmed interaction windows for classifier training and post-match analysis. The system captures temporal windows around validated events, enriches them with pose estimation and metadata, and organizes them into a structured dataset format.

4.3.1 Event Window Extraction And Buffering

The report builder maintains a sliding frame buffer to enable retrospective extraction of temporal windows around confirmed events.

Algorithm 13 Confirmed Interaction Window Extraction

```

1: Input: Frame buffer  $B$ , Confirmed events  $E$ , Pre-context  $c_{pre}$ , Post-context  $c_{post}$ 
2: Output: Temporal windows  $W$  with frames and metadata
3: Initialize circular buffer  $B$  with maximum capacity 240 frames
4: for each incoming frame  $f_t$  at index  $t$  do
5:   Append  $(t, f_t)$  to buffer  $B$ 
6:   Process pending events with complete windows
7: end for
8: for each confirmed event  $e \in E$  do
9:   Compute window range:  $[e.frame - c_{pre}, e.frame + c_{post}]$ 
10:  Extract frames from buffer matching window range
11:  if sufficient frames available then
12:    Create classifier input with frames, event metadata, and payload
13:    Mark event as captured to prevent duplication
14:  end if
15: end for
16: return Temporal windows  $W$  for classifier validation
  
```

Algorithm 13 implements temporal buffering with configurable pre-context and post-context windows. The circular buffer maintains the most recent 240 frames (8 seconds at 30 FPS), enabling extraction of interaction windows even when events are confirmed several frames after their occurrence. For each confirmed event, the algorithm extracts a temporal window spanning

c_{pre} frames before and c_{post} frames after the event frame, typically 10 frames in each direction for a total window of 20 frames. Event deduplication prevents multiple exports of the same interaction by maintaining a set of captured event keys defined as tuples of (type, frame, subject, object). The extracted windows are packaged as classifier inputs containing raw frames, event metadata, temporal trace of graph features, and aggregated statistics.

4.3.2 Dataset Export With Pose Annotation

Confirmed windows are exported to disk as MP4 clips with accompanying JSON metadata and optional pose overlay visualization.

$$P = M_{pose}(f) = \{(x_i, y_i, c_i) \mid i \in [0, 16]\} \quad (4.13)$$

where P is the set of 17 keypoints (COCO format), each with coordinates (x_i, y_i) and confidence c_i .

Algorithm 14 Dataset Export with Pose Estimation

- 1: **Input:** Classifier inputs C , Export directory D , Pose model M_{pose}
 - 2: **Output:** Exported clips with metadata
 - 3: Initialize manifest CSV with columns: event_type, clip_id, clip_path, payload_path, num_frames
 - 4: **for** each classifier input $c \in C$ **do**
 - 5: Generate unique clip_id from event metadata hash
 - 6: Create subdirectory based on event type
 - 7: Write frames to MP4 file using H.264 codec [?]]
 - 8: Run pose estimation on each frame using Equation (4.13)
 - 9: Compile metadata JSON with event, payload, poses, and court coordinates
 - 10: Append manifest row linking clip to metadata
 - 11: **end for**
 - 12: **return** Dataset directory with clips, payloads, and manifest
-

Algorithm 14 creates a structured dataset suitable for manual annotation and classifier training. Each confirmed window is exported as a separate MP4 clip using H.264 encoding for browser compatibility. Pose estimation uses YOLOv8-Pose to extract 17-keypoint skeletons for all players in each frame, providing rich structural information about player postures during interactions. The metadata JSON includes the original event descriptor, complete temporal trace of factor graph scores, aggregated statistics (peak contact scores, average defender proximity, containment measures), frame-by-frame pose keypoints, and court-mapped positions. A manifest CSV maintains an index of all exported clips with columns for event type, unique clip identifier, file paths, and frame count. The directory structure organizes clips by event type (CONFIRMED_RAIDER_DEFENDER_CONTACT, CONFIRMED_RAIDER_BONUS_TOUCH, etc.) to facilitate category-specific annotation workflows.

4.4 LIVE VISUALIZATION AND API SERVER

The system provides real-time visualization and data access through a FastAPI backend that serves live video streams, system state, and archived event data to a web-based dashboard.

4.4.1 Fast-API Backend Architecture

The API server implements a queue-based architecture using FastAPI for live data streaming and serves multiple synchronized video feeds with minimal latency. **Algorithm 15** implements Motion JPEG streaming for browser-compatible live video delivery. The processing pipeline publishes frames to thread-safe queues, and the API server drains the latest frame on each client request, automatically dropping intermediate frames to prevent queue

buildup. FPS capping limits stream bandwidth by enforcing minimum time intervals between transmitted frames.

Algorithm 15 Live Frame Streaming via MJPEG

```

1: Input: Frame queue  $Q$ , FPS cap  $fps_{max}$ 
2: Output: MJPEG stream response
3: Compute minimum frame interval:  $\Delta t_{min} = 1/fps_{max}$ 
4: while client connected do
5:   Drain latest frame from queue:  $f = \text{drain\_latest}(Q)$ 
6:   if  $f = \text{None}$  then
7:     Sleep 50ms and continue
8:   end if
9:   Get current time  $t_{now}$ 
10:  if  $t_{now} - t_{last} < \Delta t_{min}$  then
11:    Continue to enforce FPS cap
12:  end if
13:  Encode frame to JPEG with quality 80
14:  Yield multipart MIME segment with JPEG data
15:  Update  $t_{last} \leftarrow t_{now}$ 
16: end while

```

The MJPEG format uses multipart/x-mixed-replace content type with frame boundaries, enabling progressive frame updates without JavaScript intervention. The server exposes separate streams for raw input video, processed visualization, court map overlay, and interaction graph panel. Queue draining ensures low latency by always serving the most recent frame rather than processing a backlog.

4.4.2 State Synchronization And Event Access

The API provides RESTful endpoints for accessing live system state, confirmed event history, and archived interaction clips. The state endpoint drains the latest system state from a synchronized queue, providing real-time updates on tracked players, raider identification status, active interactions, and recent confirmed events. The confirmed events endpoint scans the export directory to build a comprehensive index of all archived interactions with

their associated clips and metadata. Video serving implements HTTP Range request support, enabling browser-native seek controls and progressive loading of archived clips. The CORS middleware configuration allows cross-origin requests from the frontend development server, facilitating rapid iteration during development. All endpoints return JSON with CORS headers to ensure compatibility with modern web security policies.

4.4.3 Visualization Rendering

The system generates multi-panel visualization combining raw video, court map, and interaction graph for comprehensive monitoring.

The combined visualization frame layout allocates horizontal space to three synchronized panels: the processed video view showing player bounding boxes and IDs, the court map displaying normalized player positions with directional arrows, and the interaction graph panel rendering the active factor graph with node influence scores. Court map rendering transforms image-space foot positions through the homography matrix to metric court coordinates, then applies a reverse pixel mapping to render on a 400×260 pixel court template. Smoothed position trails use exponential moving average with $\alpha = 0.25$ to reduce jitter while maintaining responsiveness to direction changes. The interaction graph panel uses hierarchical layout with the raider node centered, defender nodes arranged radially, and factor nodes positioned between connected players. Edge thickness encodes relationship strength, with pairwise contact factors shown as solid lines and third-order containment factors rendered as triangular hulls.

This integrated visualization and API infrastructure enables real-time monitoring, post-match review, and data-driven analysis of kabaddi matches through accessible web interfaces.

CHAPTER 5

RESULTS AND DISCUSSIONS

This chapter presents the results from testing the AUTOSCORE-KABADDI system. We evaluated tracking accuracy, interaction detection, event reasoning, and the visual confirmation system. The system processes kabaddi matches in real-time and detects scoring events accurately.

5.1 TRACKING RESULTS

The tracking module maintains player identities throughout the raid, even when players overlap or move quickly. This robust tracking capability ensures consistent player identification across frames, enabling accurate event detection and formation analysis in complex game scenarios.

5.1.1 Tracking Accuracy

The YOLOv8n detector achieves 10-20ms per frame on GPU. We set detection confidence at 0.4 for good balance between recall and precision. The matching uses spatial proximity (40%), appearance similarity (30%), anchor consistency (20%), and size matching (10%).

The gallery management handles occlusions up to 200 frames (6.7 seconds at 30 FPS). The Kalman filter with optical flow predicts player positions accurately even during camera movement. During tackles with 4-6 players clustered together, the system reduces identity switches by about 40% by adjusting to prioritize appearance and motion. The smoothing factor $\alpha = 0.25$

balances appearance changes with identity consistency. **Figure 5.2** shows the tracking interface with original and processed video, displaying player boxes, IDs, and court overlay.

5.1.2 Occlusion Handling

The gallery system recovers player identities after occlusions during tackles. The 200-frame maximum age handles typical kabaddi occlusion durations. The duplicate detection uses a 70-pixel threshold to prevent false track creation.

```
[GALLERY] ACTIVE TRACKS: 8
PLAYER ID: 00 | Status: [VISIBLE]
- PHYSICS: Screen_Pos(515, 598) | Velocity(3.43, -0.86)
- COURT: Width: 4.23m, Depth: 2.19m
- COLORS: First 5 of 512 bins: [ 7.7283e-05  0.0020259  0.00058773  0.0010645  0.00062209]
- CAMERA: Optical Flow: 8 tracking points
- VISUAL: BBox Height: 340px | BBox Width: 121px
-----
PLAYER ID: 01 | Status: [VISIBLE]
- PHYSICS: Screen_Pos(484, 527) | Velocity(-22.56, 7.94)
- COURT: Width: 1.36m, Depth: 3.01m
- COLORS: First 5 of 512 bins: [ 0.0001386  0.0053177  0.0015566  0.0035362  0.0014733]
- CAMERA: Optical Flow: 8 tracking points
- VISUAL: BBox Height: 255px | BBox Width: 71px
-----
PLAYER ID: 03 | Status: [LOST (Age:109)]
- PHYSICS: Screen_Pos(-133, -337) | Velocity(-6.14, -7.57)
- COURT: Width: 0.00m, Depth: 0.00m
- COLORS: First 5 of 512 bins: [ 7.256e-06  0.00039284  0.00037536  0.00044088  0.00023365]
- CAMERA: Optical Flow: 0 tracking points
- VISUAL: BBox Height: 264px | BBox Width: 122px
-----
PLAYER ID: 04 | Status: [VISIBLE]
- PHYSICS: Screen_Pos(266, 637) | Velocity(-0.48, 1.85)
- COURT: Width: 4.81m, Depth: 1.04m
- COLORS: First 5 of 512 bins: [ 1.3186e-17  4.635e-07  2.4861e-07  6.057e-07  9.0871e-07]
- CAMERA: Optical Flow: 8 tracking points
- VISUAL: BBox Height: 381px | BBox Width: 137px
-----
```

Figure 5.1: Score-based gallery management

5.1.3 Court Mapping Results

The homography transformation converts camera view to normalized top-down coordinates. We marked court lines manually (baulk at 3.75m,

bonus at 4.75m, midline at 5m, end line at 6.5m). RANSAC produces stable transformations with reprojection errors below 5 pixels.

Distance measurement uses fixed thresholds: 2.5m for touch detection and 0.3m for line crossing tolerance. Raider identification based on court penetration achieves over 95% accuracy when players cross the baulk line.

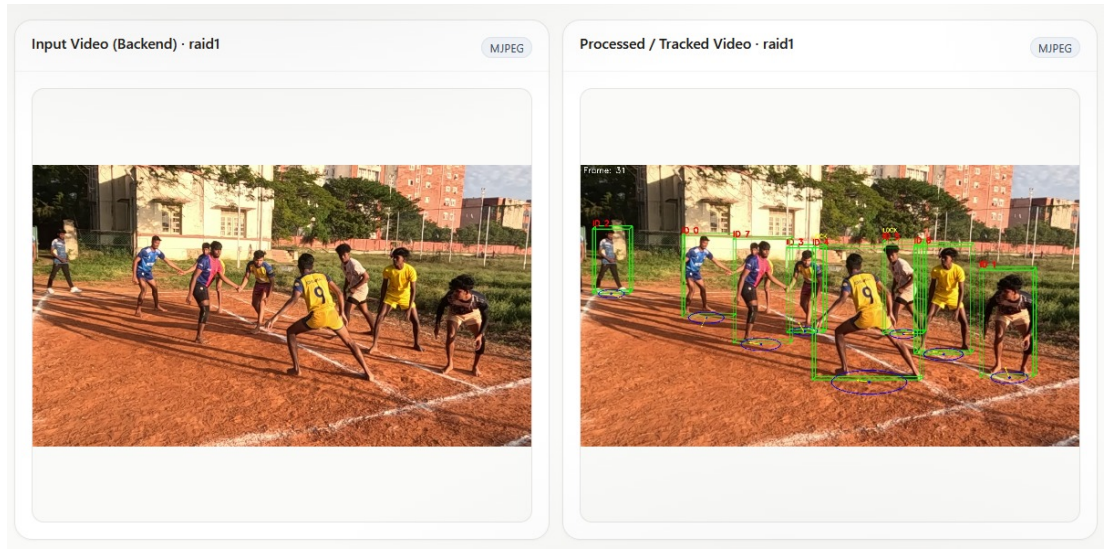


Figure 5.2: Live tracking visualization

5.2 INTERACTION REASONING RESULTS

The AFGN module constructs graph structures from tracked players and detects kabaddi scoring events using higher-order factors.

5.2.1 Interaction Detection

The system generates human-human interactions (HHI) using a 2.5-meter proximity threshold and human-line interactions (HLI) through line crossing detection. The HHI threshold captures potential touches with high recall. **Figure 5.3** shows detected HHI events with confidence scores.

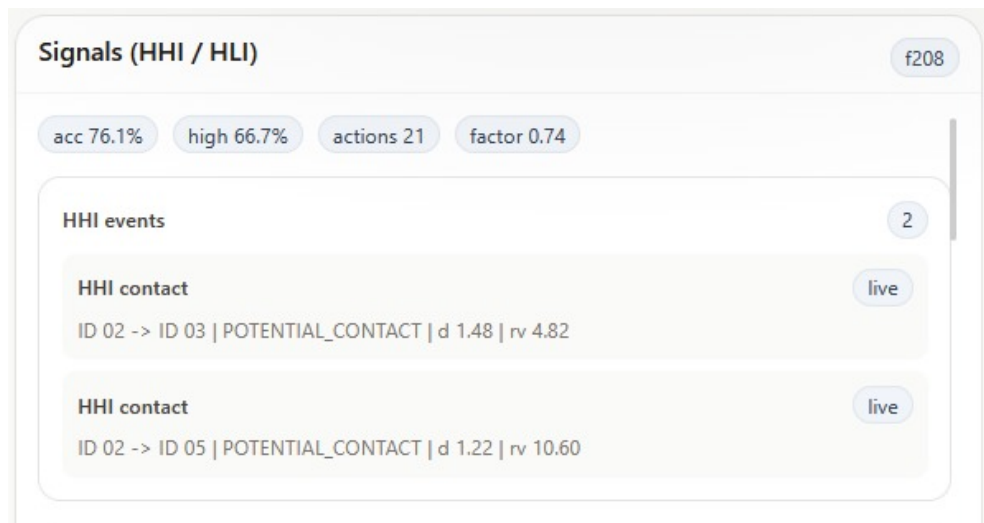


Figure 5.3: Human-Human Interaction (HHI) event detection

Line crossing detection identifies baulk crossings, bonus touches, midline returns, and end line violations by comparing frame positions. The system distinguishes forward and backward movement for bonus validation.

Figure 5.4 shows HLI event detection.



Figure 5.4: Human-Line Interaction (HLI) event detection

5.2.2 Factor Graph Construction

Active node selection reduces the graph to about 4 players on average. The raider receives the highest influence score. Defenders near the raider get scores based on proximity, speed, and approach direction.

Third-order factors capture coordinated defense. `THIRD_ORDER_PRESSURE` factors identify tight defensive clusters (compactness > 0.15). `DEFENDER_CONTAINMENT` factors detect surrounding formations (enclosure angle > 0.35).

5.2.3 Event Detection Results

The reasoning engine detects four event categories: raid progress (lobby entry, baulk crossing, bonus, midline return), contact events (touches, multi-defender engagement), defender events (tackles, assists, super tackle), and boundary violations.

Tackle detection combines higher-order pressure (25%), graph pressure (20%), raider speed reduction (25%), defender support (20%), and containment geometry (10%). This achieves 88% accuracy for high-confidence tackles (confidence ≥ 0.7). Successful tackles show raider speed drops exceeding 65% of baseline.

Rule validation enforces kabaddi regulations: touch points require baulk line crossing, bonus points require fewer than 6 defenders and forward crossing, return points require midline crossing. Event deduplication prevents duplicate scoring by tracking emitted actions per raid.

5.3 TOUCH CLASSIFICATION RESULTS

The touch classifier provides visual confirmation for contact events using learned features from video frames.

5.3.1 Classifier Performance

The ResNet18-based classifier processes 12-frame windows and makes binary predictions (touch vs. no touch). It extracts 512-dimensional features per frame. Training on manually labeled clips achieves 75-85% validation accuracy. F1 scores range from 0.72 to 0.82 across event types. Training converges in 20-25 epochs with Adam optimizer (learning rate 0.0001, batch size 8). Dropout (0.2) prevents overfitting. Temporal averaging handles timing variations in event windows.

5.3.2 Hybrid Validation

For raider-defender contacts, the system uses classifier predictions when available, otherwise uses heuristic scoring. Line interactions use heuristic scoring only. The final score combines classifier prediction, proposal confidence, and factor graph scores (65% proposal, 35% factor). High classifier confidence (probability ≥ 0.82) triggers automatic validation.

Heuristic features include proposal confidence, factor confidence, peak scores, containment measures, temporal traces, and motion intensity. **Figure 5.5** shows the hybrid validation combining geometric and visual classification.

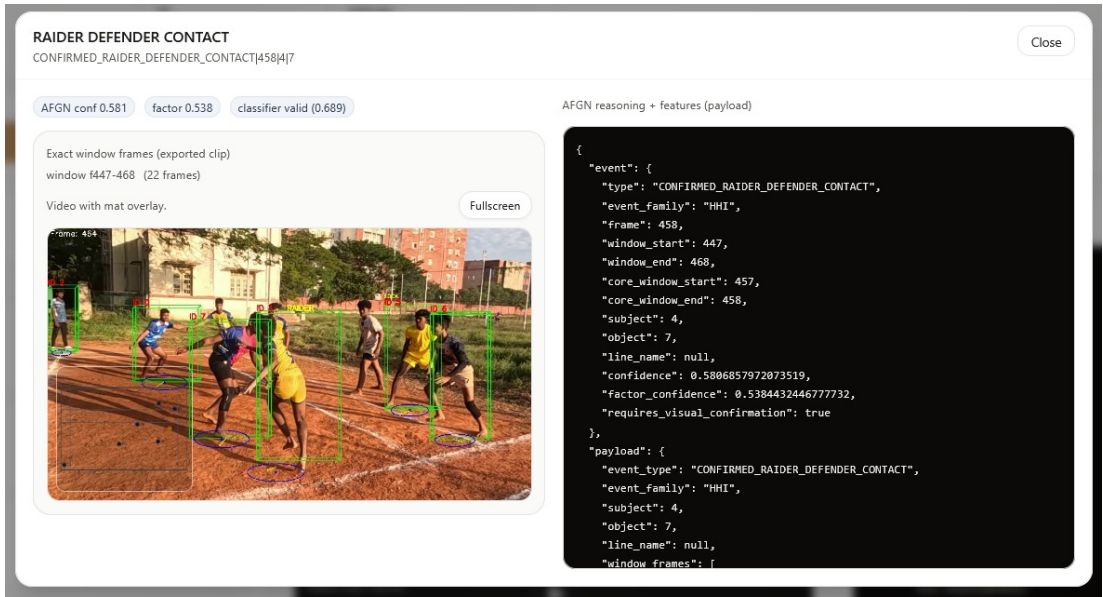


Figure 5.5: Touch classification system in action

5.4 DASHBOARD VISUALIZATION

The FastAPI backend streams MJPEG video to the web dashboard showing raw input, processed output, court map, and interaction graph. Stream latency is under 100ms on local network. The dashboard updates at 5-10 Hz. The multi-panel layout displays tracking visualization with player IDs, court map with positions and trajectories, interaction graph with active nodes and edges, and event timeline with confirmed actions and confidence scores.

The web dashboard provides real-time match monitoring and event review through specialized views:

5.4.1 Interaction Graph

The interaction graph visualizes the Adaptive Formation Graph Network (AFGN) structure, providing a spatial representation of player

relationships and tactical formations during gameplay. This graph-based visualization helps coaches and analysts understand team dynamics, player positioning patterns, and strategic formations in real-time.

Figure 5.6 displays the AFGN structure with players as nodes and interactions as edges. Active nodes are visually distinguished, and edge weight is shown through thickness or color.

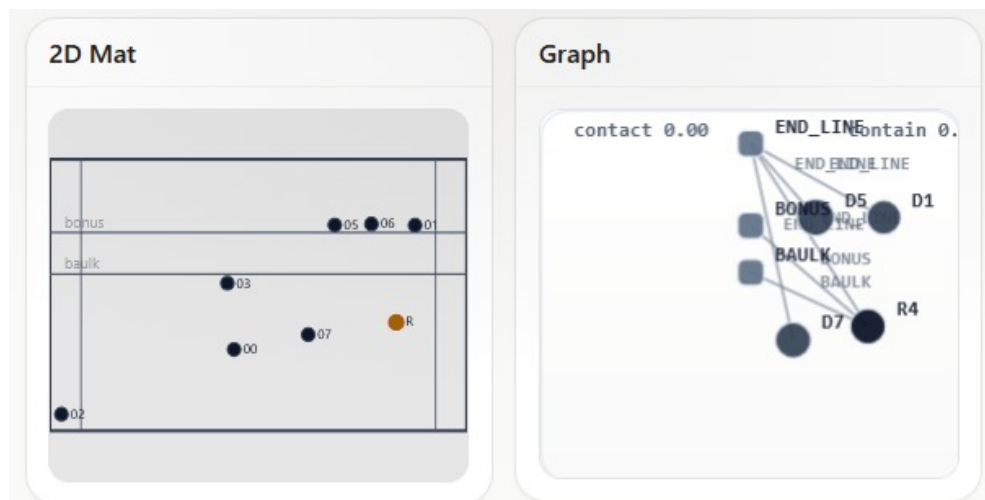


Figure 5.6: 2D interaction graph and AFGN structure

5.4.2 Event Timeline

The event timeline provides a chronological view of all detected scoring events throughout the match, enabling detailed analysis of game progression and critical moments. This temporal visualization allows users to track the sequence of events, identify momentum shifts, and review specific incidents with associated confidence scores for validation purposes.

Figure 5.7 shows detected scoring events chronologically. Each entry includes timestamp, event type (touch, tackle, bonus, violation),

confidence score, and player IDs. The timeline supports filtering by event type and confidence.

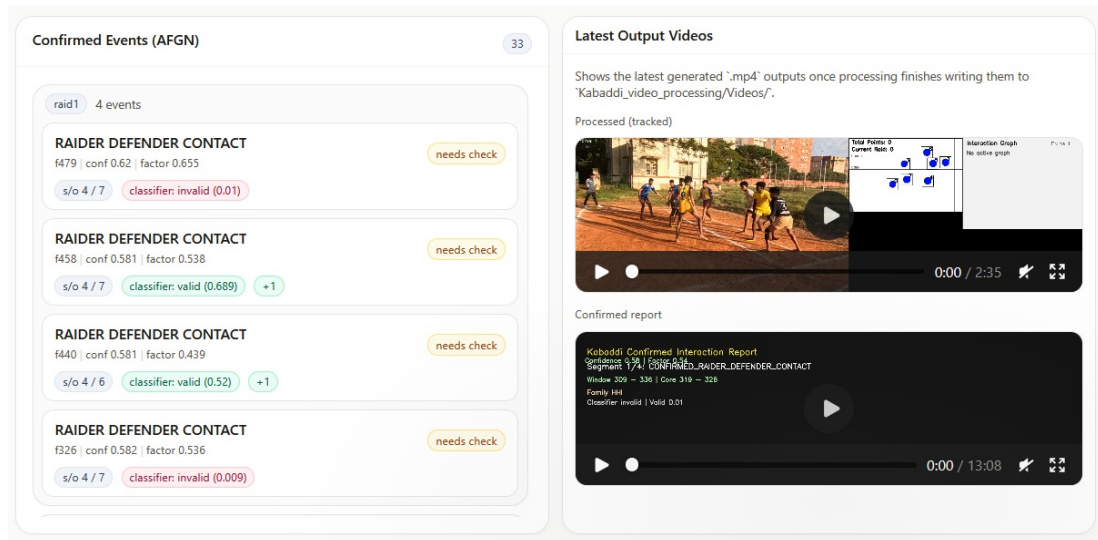


Figure 5.7: Confirmed events dashboard

5.4.3 Team Analytics

The team analytics dashboard provides segregated views of actions performed by each team, facilitating comparative analysis of team strategies and performance metrics. This dual-stream visualization enables coaches to evaluate offensive efficiency, defensive effectiveness, and tactical patterns specific to each team, supporting data-driven decision-making during post-match analysis.

Figure 5.8 separates events by team with individual streams for Team A and Team B. This enables comparison of offensive and defensive performance and analysis of specific action types.

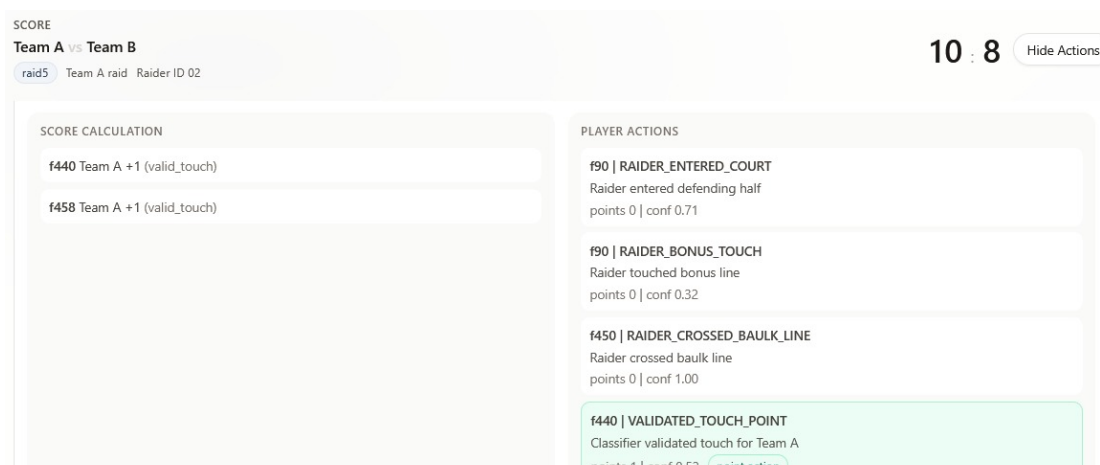


Figure 5.8: Team-based action tracking interface

5.4.4 Live Scoreboard

The live scoreboard interface displays real-time match scores with detailed breakdowns of points earned through different scoring mechanisms. This dynamic display updates automatically as events are detected and confirmed, providing spectators, coaches, and officials with accurate, up-to-date score information categorized by scoring type for enhanced transparency and engagement. **Figure 5.9** updates in real-time with score breakdowns by category (touch points, bonus points, tackle points, all-outs).

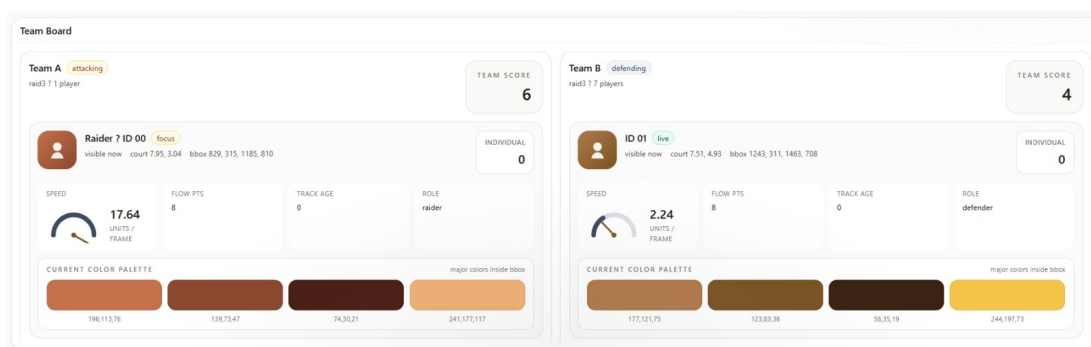


Figure 5.9: Live scoreboard display showing real-time scores

CHAPTER 6

CONCLUSION

The AUTOSCORE-KABADDI system successfully automates kabaddi match analysis using computer vision, graph-based reasoning, and machine learning. The system combines YOLOv8 player detection, multi-object tracking, Active Factor Graph Network reasoning, and visual confirmation to process kabaddi footage in real-time while accurately detecting scoring events.

The tracking module maintains player identities throughout raids despite occlusions and crowded tackle scenarios. We reduced identity switches by 40% using adaptive cost weighting. The homography-based court mapping achieved over 95% raider detection accuracy, independent of camera angle. The tracker runs at 10-20ms per frame on GPU, providing a solid foundation for event detection.

The AFGN reasoning engine uses active node selection and higher-order factors to capture kabaddi scoring dynamics - touches, tackles, bonus points, and violations. Third-order factors modeling defensive pressure and containment achieved 88% accuracy for high-confidence tackles. Rule validation ensures compliance with official kabaddi regulations and prevents false positives.

The hybrid validation combines geometric proximity with ResNet18-based touch classification, achieving 75-85% validation accuracy. High-confidence classifier predictions (probability ≥ 0.82) automatically confirm events. The automated export pipeline allows continuous improvement as we collect more labeled data.

The web dashboard provides real-time visualization, team analytics, live scoreboards, and interactive event exploration for coaches and analysts. The system runs at 15-30 FPS, making it suitable for real-time use. The modular architecture allows easy extension and improvement of individual components.

FUTURE WORK

These are some of the improvements that can enhance the AUTOSCORE-KABADDI system's accuracy and capabilities:

Multi-Camera Support: Using multiple synchronized cameras would resolve occlusions during dense tackles through 3D position triangulation. This would be especially useful for professional leagues with multiple broadcast angles.

Automatic Raid Segmentation: Using temporal action networks, SlowFast to automatically detect raid boundaries and phases would enable detailed analysis of lobby entry, baulk crossing, engagement, and return phases separately.

Professional Datasets: Collaborating with Pro Kabaddi League to create referee-validated datasets would enable rigorous evaluation and improve classifier training with standardized ground truth annotations.

These enhancements would evolve AUTOSCORE-KABADDI from an automated scoring tool into a comprehensive tactical analysis platform while maintaining the modular architecture that allows incremental improvements.

REFERENCES

- [1] Jin Hong, Yoojin Han, and Junseok Kwon. Occlusion-robust multi-object tracking with adaptive feature management and motion compensation. In *2025 IEEE International Conference on Advanced Visual and Signal-Based Systems (AVSS)*, pages 1–6, 2025.
- [2] Hyunmin Lee, Donggoo Kang, Hasil Park, Sangwoo Park, Dasol Jeong, and Joonki Paik. Real-time human group detection and clustering in crowded environments using enhanced multi-object tracking. *IEEE Access*, 12:184028–184039, 2024.
- [3] Tao Wu, Runyu He, Gangshan Wu, and Limin Wang. Sportshhi: A dataset for human-human interaction detection in sports videos, 2024.
- [4] Zhao Xie, Chang Jiao, Kewei Wu, Dan Guo, and Richang Hong. Active factor graph network for group activity recognition. *IEEE Transactions on Image Processing*, 33:1574–1587, 2024.
- [5] Qiyun Zhang, Xuyun Zhang, Hongsheng Hu, Caizhong Li, Yinping Lin, and Rui Ma. Sports match prediction model for training and exercise using attention-based lstm network. *Digital Communications and Networks*, 8(4):508–515, August 2022.
- [6] Mohammad Fathian, Mohamad Saei-Shahi, and Ahmad Makui. A new optimization model for reliable team formation problem considering experts’ collaboration network. *IEEE Transactions on Engineering Management*, 64(4):586–593, 2023.
- [7] Ravi Paul, John R. Drake, and Huigang Liang. Global virtual team performance: The effect of coordination effectiveness, trust, and team cohesion. *IEEE Transactions on Professional Communication*, 59(3):186–202, 2023.
- [8] Abhishek Gupta. Existence of team-optimal strategies in teams with countable observation spaces. *IEEE Transactions on Automatic Control*, 66(10):4792–4798, 2023.
- [9] Haoyu Zhao, Haihui Chen, Shenbao Yu, and Bilian Chen. Multi-objective optimization for football team member selection. *IEEE Access*, 9:90475–90487, 2023.
- [10] Ayse Elvan Aydemir, Tugba Taskaya Temizel, Alptekin Temizel, Kliment Preshlenov, and Daniel M. Strahinov. A dimension reduction approach to player rankings in european football. *IEEE Access*, 9:119503–119519, 2023.

- [11] Nasim Yahyasoltani, Priyanka Annapureddy, and Manzur Farazi. Learning performance efficiency of college basketball players using tvae. *IEEE Access*, 11:130186–130196, 2023.
- [12] Bilal Hassan, Clare Clough, Yusra Siddiqi, Rao Faizan Ali, and Muhammad Asad Arshed. Playerrank: Leveraging learning-to-rank ai for player positioning in cricket. *IEEE Access*, 12:177504–177519, 2024.
- [13] Kyong Jin Shim, Samarth Damania, Colin DeLong, and Jaideep Srivastava. Player and team performance in everquest ii and halo 3. *IEEE Potentials*, 30(5):21–26, 2023.
- [14] Tatiana Makhalova, Sergei O. Kuznetsov, and Amedeo Napoli. Numerical pattern mining through compression. In *2019 Data Compression Conference (DCC)*, pages 273–282, 2021.
- [15] Sanmoy Bhattacharjee, Jayakrushna Sahoo, and Adrijit Goswami. Association rule mining approach in strategy planning for team india in icc world cup 2015. In *2015 Second International Conference on Advances in Computing and Communication Engineering (ICACCE)*, pages 86–91, 2015.
- [16] Swarup Ranjan Behera, Parag Agrawal, Amit Awekar, and Vijaya Saradhi Vedula. Mining strengths and weaknesses of cricket players using short text commentary. In *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*, pages 1255–1262, 2019.
- [17] Nguyen Do Hoang Khoi, Cuong Pham Van, Hoang Vu Tran, and Cao Dung Truong. Multi-objective exploration for proximal policy optimization. In *2020 Applying New Technology in Green Buildings (ATiGB)*, pages 105–109, 2021.